

SQIsignTriangle

Short **Q**uaternion and **I**sogeny **signature** with a **Triangle**-shaped framework

Principal Submitter: Wei Yu^{1,2}, +86-15210641736, yuwei@iie.ac.cn

Contributors:

- **Design:** Bin Wang^{1,2}, wangbin6616@iie.ac.cn

Jiaxuan Qu^{1,2}, qujiaxuan@iie.ac.cn

Song Tian^{1,2}, tiansong@iie.ac.cn

- **Analysis:** Wenbiao Chen^{1,2}, chenwenbiao@iie.ac.cn

Kunpeng Wang^{1,2}, wangkunpeng@iie.ac.cn

- **Implementation (X86):** Tingyu Xing^{1,2}, xingtingyu@iie.ac.cn

Xiuyu Qiu^{1,2}, qiuxiuyu@iie.ac.cn

Organizations: 1. State Key Laboratory of Cyberspace Security Defense,

Institute of Information Engineering, CAS

2. University of Chinese Academy of Sciences

Owner: Wei Yu

Postal Address: No. 19 Shucun Road, Haidian District, Beijing, 100085, China

Contact: chenwenbiao@iie.ac.cn, +86-13121977714

Contents

1	Introduction	4
1.1	Overview of SQIsignTriangle	4
1.2	Advantages and disadvantages	5
1.3	Outline	6
1.4	Notation	7
2	Preliminaries	9
2.1	Elliptic curves	9
2.1.1	Montgomery curves	9
2.1.2	The group law	10
2.1.3	Torsion subgroups and deterministic basis computation	11
2.2	Abelian varieties and Isogenies	11
2.2.1	Isogenies between elliptic curves	12
2.2.2	Isogenies between abelian varieties	13
2.3	Quaternions and ideals	15
2.3.1	Quaternion orders and ideals	16
2.3.2	Basic operations on ideals.	16
2.4	Converting between ideals and isogenies	17
2.4.1	Deuring Correspondence.	17
2.4.2	Qlapoti: converting $\mathcal{O}_0$ -ideals to isogenies.	18
3	Algorithms Description	21
3.1	Elliptic curve arithmetic	23
3.2	Quaternions and ideals	26
3.3	Isogenies	33
4	The Signature Scheme	36
4.1	Parameters	36
4.2	Key generation	36
4.3	Signing	37
4.4	Verification	38
5	Design Rationale	40
5.1	Σ -protocols and Fiat-Shamir technique	40
5.2	Σ -protocol of SQIsignTriangle	41
5.3	Hardness assumption and parameter choices	42
5.4	Failure Cases	43
6	Security Statements and Analyses	48
6.1	Hint Distribution and Hardness Assumptions	48
6.2	Statistical and Knowledge Properties of the Σ -Protocol	49
6.3	Hint-Assisted Weak Honest-Verifier Zero Knowledge	51

6.4	EUF-CMA Security	53
7	Performance Evaluation	55
7.1	Design-Level Performance Analysis	55
7.2	Parameter Selection	56
7.3	Key and Signature Sizes	57
7.4	Implementation Platforms	57
7.5	Software Performance	58
7.6	Software Resource Consumption	58
8	Feature Statements	59

List of Figures

1	A description of the Σ -protocol	4
2	A (d_1, d_2) -isogeny diamond	14
3	Qlapoti isogeny diamond	19
4	A description of response stage	36
5	A description of the Σ -protocol	42

List of Tables

1	SQIsignTriangle key and signature sizes in bytes for each security level	6
2	SQIsignTriangle performance in 10^6 CPU cycles for the optimized 64-bit Intel implementation on Intel Core i7-13700 (2.10 GHz). Results are averaged over 10 benchmark runs	6
3	Notation and parameters of the SQIsignTriangle scheme.	8
4	Deuring Correspondence	18
5	Implementation parameters used in our algorithms.	23
6	SQIsignTriangle parameter sets	43
7	Design-level performance drivers of SQIsignTriangle	56
8	SQIsignTriangle parameter sets inherited from SQIsign.	56
9	SQIsignTriangle encoded key and signature sizes in the submitted C API.	57
10	SQIsignTriangle optimized C performance in 10^6 CPU cycles on Intel Core i7-13700. Results are over 10 benchmark runs.	58
11	SQIsignTriangle software resource consumption measured with <code>/usr/bin/time -v</code>	58

1 Introduction

This document presents a signature scheme called SQIsignTriangle, whose security relies on the hardness of computing the endomorphism ring of random supersingular elliptic curves. This scheme is a variant of SQIsign [AAA⁺25].

1.1 Overview of SQIsignTriangle

SQIsignTriangle is designed as a Σ -protocol, turned into a signature scheme by the Fiat-Shamir [FS87] technique. The Σ -protocol is three-round and assumes the hardness of computing endomorphism ring of supersingular elliptic curves which is the fundamental problem in isogeny-based cryptography. The Σ -protocol works as follows and the sketch picture is shown in Figure 1:

1. The prover generates the secret isogeny $\varphi_{\text{sk}} : E_0 \rightarrow E_{\text{pk}}$, the public key is $\text{pk} = E_{\text{pk}}$ and the secret key is $\text{sk} = \varphi_{\text{sk}}$.
2. The prover generates the isogeny $\varphi_{\text{com}} : E_0 \rightarrow E_{\text{com}}$ and then sends E_{com} to the verifier as the commitment.
3. A verifier selects two integers c_1, c_2 and send them to the prover.
4. The prover computes with a two-dimensional representation of an isogeny $\varphi_{\text{rsp}} : E_{\text{pk}} \rightarrow E_{\text{com}}$ of degree q such that $q \equiv c_2 \pmod{c_1}$.
5. The verifier computes the response isogeny of degree q , then checks its correctness.

The Σ -protocol is based on the hard problem that computing endomorphism ring of supersingular elliptic curves is difficult. Given a random supersingular elliptic curve E , compute its endomorphism ring with an efficient representation. This difficult problem has been studied in isogeny-based cryptography for decades, and it is still believed that it can only be solved in exponential time. No efficient polynomial-time algorithm has been found, either in the classical setting or in the quantum setting. After having the Σ -protocol, we can use the Fiat-Shamir to construct a signature scheme. The details of the signature are described in Chapter 4.

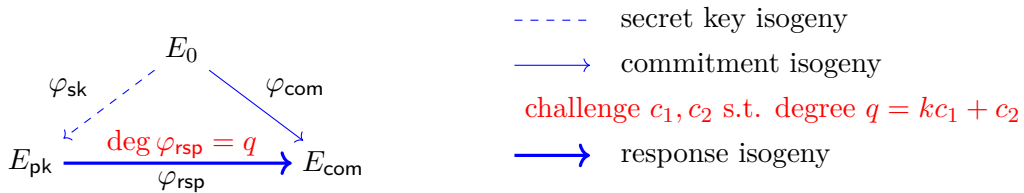


Figure 1: A description of the Σ -protocol

Other isogeny-based signatures. Currently, isogeny-based signature schemes can be classified into three categories based on the underlying hard problems: the first category is based

on the problem of computing the endomorphism ring of supersingular elliptic curves; the second category is based on the group action of supersingular elliptic curves; and the third category is based on computing large prime degree isogenies on elliptic curves. The representative scheme of the first category is SQIsign [AAA⁺25], which has also been selected for the third round of the current NIST additional signature call. This scheme has several variants, such as SQIsignHD [DLRW24], SQIsign2D-West [BDF⁺24], SQIsign2D-East [NOC⁺24], SQIsign-Prime [DF24], SQIsign2D² [XLZO25] and SQIsign2DPush [NO25]. The representative scheme of the second category is CSI-FiSh [BKV19]; however, due to issues related to quantum security, signature sizes in this category are too large and the speed is very slow. The representative scheme of the third category is PRISM [BBC⁺25]. Below we will describe the advantages and disadvantages of our scheme.

Differences. Our signature scheme is similar to SQIsign. The key generation and commitment stage are same as SQIsign. The main difference lies in the selection of the challenge space: SQIsign chooses a random 2-isogeny chain as the challenge, whereas SQIsignTriangle’s challenge is a set of features of the response isogenies or to say the response isogenies whose degrees satisfy a special relation. In terms of efficiency, we save the computation of the 2-isogeny chain, making the overall signing time slightly faster than SQIsign. Regarding signature size, our scheme is marginally larger than SQIsign. In terms of security, we prove that the signature scheme is EUF-CMA secure in the random oracle model, assuming that computing the endomorphism ring of a supersingular elliptic curve is hard.

1.2 Advantages and disadvantages

Advantages. The following aspects describe the advantages of SQIsignTriangle .

- ⊕ **Compact keys and signatures.** SQIsignTriangle offers both compact public keys and signature (see Table 1). They are at least $5\times$ smaller than lattice-based signatures (e.g., ML-DSA [Nat24a]) and at least $100\times$ smaller than hash-based signature (e.g., SLH-DSA [Nat24b]) with respect to the combined size of public key and signature.
- ⊕ **A simple signing procedure.** Compared with other isogeny-based signatures (SQIsign family), the construction of SQIsignTriangle is somewhat simple in its mathematical principles and easy to understand.
- ⊕ **Provable security.** The security proof of this scheme is clear, and the scheme is EUF-CMA secure under the hardness assumption of computing the endomorphism ring of a supersingular elliptic curve. SQIsignTriangle supports signing messages of length up to 2^{63} bits, and for a single key, it can be used to sign up to 2^{64} times.

Disadvantages. The following aspects describe the disadvantages of SQIsignTriangle .

- ⊖ **A relative new assumption.** The endomorphism ring problem has been studied since 1996 [Koh96]. Perhaps from a cryptographic perspective, it has not been studied for very long time compared with other post-quantum hardness assumption.

Table 1: SQIsignTriangle key and signature sizes in bytes for each security level

	λ	Public-key size	Signature size
SQIsignTriangle	128	65	204
	160	81	255
	256	129	408
	512	257	816

- ⊖ **Relatively slow efficiency.** In terms of efficiency, compared to lattice-based and hash-based signature schemes, SQIsignTriangle is relatively slow in key generation, signing, and verification, with its speed being one to two orders of magnitude slower (see Table 2).

Table 2: SQIsignTriangle performance in 10^6 CPU cycles for the optimized 64-bit Intel implementation on Intel Core i7-13700 (2.10 GHz). Results are averaged over 10 benchmark runs

λ	Key Gen.	Signing	Verification
128	13.9	37.3	4.0
160	21.8	64.0	7.6
256	108.5	201.3	27.7
512	783.3	2698.8	258.8

1.3 Outline

In Section 2, we recall the mathematical preliminaries used by SQIsignTriangle, including elliptic curves, isogenies, higher-dimensional isogeny techniques, quaternion ideals, and the Deuring correspondence.

In Section 3, we describe the algorithmic building blocks used in the scheme. These include elliptic-curve arithmetic, quaternion and ideal operations, and isogeny algorithms. Core routines are given as full algorithms, while standard or auxiliary subroutines are specified by their inputs, outputs, mathematical operations, and their role in the construction. This provides the computational basis for the complete description of key generation, signing, and verification.

In Section 4, we give the full specification of the SQIsignTriangle signature scheme. We first list the parameter sets for the submitted security levels, covering the required 128-bit, 256-bit, and 512-bit classical security levels and their corresponding quantum security targets. We then describe the key generation, signing, and verification algorithms. This section also specifies the digital-signature functionality of the scheme: SQIsignTriangle consists of **KeyGen**, **Sign**, and **Verify**, supports messages of length up to 2^{63} bits, and is intended to support signing and verification for at least 2^{64} distinct messages under a single key pair.

In Section 5, we explain the design rationale of SQIsignTriangle. We describe the underlying Σ -protocol, the Fiat–Shamir transformation, and the modular-degree challenge structure used in the scheme. We then discuss the mathematical hardness assumptions, the parameter-selection rationale for the submitted instances, and the known classical and quantum attack complexities for each declared security level. We also analyze the relevant failure cases in the signing procedure and explain how their probabilities affect correctness and security.

In Section 6, we present the formal security analysis of SQIsignTriangle. We establish hint-assisted weak honest-verifier zero knowledge and derive EUF-CMA security in the random-oracle model, relying on the hardness of the One Endomorphism problem with hints and a hint indistinguishability assumption.

In Section 7, we evaluate the performance of SQIsignTriangle. We first give a design-level performance analysis, identifying the dominant costs in key generation, signing, and verification. We then discuss parameter selection, encoded public-key, secret-key, and signature sizes, and implementation platforms. Finally, we report software performance and resource consumption for the submitted implementation, including implementation details and benchmarks on a mainstream 64-bit PC platform.

In Section 8, we summarize the main features of SQIsignTriangle. We highlight its innovative modular-degree challenge structure, compact key and signature sizes, EUF-CMA security based on the Fiat–Shamir-with-hints paradigm, simple signing structure, constant-time implementation potential, and compatibility with SQIsign-oriented software and hardware environments.

1.4 Notation

In Table 3, we list all the specific symbols that will be employed throughout this technical report. The notation used here is adopted from the SQIsign [AAA⁺25] and SIKE [JAC⁺22] documents, and also aligns with our common practice in daily research.

Table 3: Notation and parameters of the SQIsignTriangle scheme.

Scheme Parameters	
λ	Security parameter, one of 128, 160, 256, or 512.
p	Size of the base field, a prime of the form $c \cdot 2^f - 1$.
f	The 2-valuation of $p + 1$, roughly 2λ .
Elementary Objects	
$\mathbb{Z}/d\mathbb{Z}$	Ring of integers modulo d for some integer d .
$\mathbb{F}_p$	Finite fields with p elements.
$\mathbb{F}_q$	Finite fields with $q = p^k$ elements.
$\mathbb{F}_{p^2}$	Finite fields with p^2 elements.
$\overline{\mathbb{F}}_p$	Algebraic closure of $\mathbb{F}_p$.
Elliptic Curve	
E	An elliptic curve over some field $\mathbb{F}_q$.
$E \times E'$	A product of two elliptic curves E and E' .
A	An abelian variety over some field $\mathbb{F}_q$.
$\text{Jac}(C)$	The Jacobian of a hyperelliptic curve C .
$E_{A,B}$	An elliptic curve in Montgomery form with parameters $A, B \in \mathbb{F}_q$.
0_E	The point at infinity of an elliptic curve E .
$E(\mathbb{F}_q)$	The points on E with coordinates in $\mathbb{F}_q$.
$E[n]$	The n -torsion on E .
hint_i	Hints to generate a basis for $E[2^f]$.
$j(E)$	The j -invariant of the elliptic curve E .
x_P and y_P	The x -resp. y -coordinate of a point $P \in E$.
$\varphi : E \rightarrow E'$	An isogeny between elliptic curves.
$\Phi : A \rightarrow A'$	A higher-dimensional isogeny.
$\text{End}(E)$	The endomorphism ring of an elliptic curve E .
Quaternion Objects	
$B_{p,\infty}$	A quaternion algebra, ramified at p and ∞ .
$B_{p,\infty}^*$	The space of linear functions $B_{p,\infty} \rightarrow \mathbb{Q}$.
$\mathcal{O}$	An order in $B_{p,\infty}$.
$\mathcal{O}_L(I), \mathcal{O}_R(I)$	The left, resp. right, order of an ideal $I \subset \mathcal{O}$.
$\text{tr}(\alpha)$	The trace of an element $\alpha \in B_{p,\infty}$.
$\text{nrd}(\alpha)$	The norm of an element $\alpha \in B_{p,\infty}$.
$\text{nrd}(I)$	The norm of an ideal $I \subset \mathcal{O}$.
$[I]^*$	The pullback by an ideal I .
$[I]_*$	The pushforward by an ideal I .
Protocol Objects	
pk	The public key.
sk	The secret key.
msg	The message to be signed.
hash	A hash function, obtained by SHAKE256.
D_{mix}	Degree of the secret isogeny, chosen as the smallest prime larger than $2^{4\lambda}$.
c_1, c_2	The challenge.

2 Preliminaries

SQIsignTriangle relies on a deep mathematical correspondence between two seemingly unrelated domains. On the one hand, it involves supersingular **elliptic curves** and **isogenies** defined over **finite fields**, encompassing both the one-dimensional case and its higher-dimensional generalizations. On the other hand, it is connected to **maximal orders** and **ideals** in **quaternion algebras**.

To fully implement SQIsignTriangle, one must be able to represent these mathematical structures and carry out computations with them. In this chapter, we review the necessary preliminaries that form the foundation of SQIsignTriangle.

2.1 Elliptic curves

In the following, we assume that $\mathbb{F}_q$ is a finite field with $\text{char}(\mathbb{F}_q) > 2$. Elliptic curves are curves of genus one having a specified base point. We recall several key facts on supersingular elliptic curves in Montgomery form that are required for the implementation of SQIsignTriangle. For a comprehensive overview of Montgomery curves and their properties, we refer the reader to [CS18].

2.1.1 Montgomery curves

Let $A, B \in \mathbb{F}_q$ with $B(A^2 - 4) \neq 0$. The Montgomery curve over $\mathbb{F}_q$ associated with the parameters A and B is the elliptic curve

$$E_{A,B} : By^2 = x^3 + Ax^2 + x.$$

Its points consist of all pairs (x, y) satisfying this equation over $\mathbb{F}_q$ and its extension fields, together with the point at infinity 0_E . When it is necessary to indicate the ground field explicitly, we write $E_{A,B}/\mathbb{F}_q$. In the normalized case $B = 1$, we abbreviate $E_{A,B}$ to E_A , and use E to denote a Montgomery curve when the parameters are not specified. The subset of points rational over $\mathbb{F}_q$ is denoted by $E_{A,B}(\mathbb{F}_q)$, and the parameter A is usually called the *Montgomery coefficient*.

Two Montgomery curves over $\mathbb{F}_q$ are said to be *isomorphic* if one is obtained from the other by $(x, y) \mapsto (Dx + R, Cy)$ with $D, C, R \in \mathbb{F}_q$. When such a transformation does not exist over $\mathbb{F}_q$ itself, but does exist after passing to a finite extension field, the two curves are regarded as *twists* of one another. A basic instance occurs for curves $E_{A,B}$ and $E_{A,B'}$: these form a pair of *quadratic twists*, arising through the choice $C = \sqrt{B/B'}$, and they become isomorphic over $\mathbb{F}_q$ exactly in the case where B/B' is a square in $\mathbb{F}_q$.

Let N denote the number of points on $E_{A,B}$, including 0_E . The curve is said to be *supersingular* when $N \equiv 1 \pmod{\text{char}(\mathbb{F}_q)}$. In this work, we focus on supersingular curves defined over $\mathbb{F}_{p^2}$ with $p \equiv 3 \pmod{4}$. In this setting, any supersingular curve $E_A/\mathbb{F}_{p^2}$ (with $B = 1$) has exactly $(p+1)^2$ points. Its quadratic twists $E_{A,\gamma}/\mathbb{F}_{p^2}$, where γ is a quadratic non-residue in $\mathbb{F}_{p^2}$, have $(p-1)^2$ points and are all mutually isomorphic.

Isomorphisms between Montgomery curves. Associated with a Montgomery curve $E_{A,B}$ is its j -invariant, defined by

$$j(E_{A,B}) = \frac{256(A^2 - 3)^3}{A^2 - 4}.$$

Over an algebraic closure, the j -invariant classifies elliptic curves up to isomorphism and quadratic twisting: two curves have the same j -value exactly when they are either isomorphic or related by a quadratic twist. Assume now that the two Montgomery curves $E_{A,B}$ and $E_{A',B'}$ are isomorphic. Then an isomorphism sending the first curve to the second, as described above, can be written in the form $(x, y) \mapsto (Dx + R, Cy)$, where

$$D = \lambda^2 \frac{B'}{B}, \quad R = \lambda^2 \frac{AB'}{3B} - \frac{A'}{3}, \quad C = \lambda^3 \frac{B'}{B}, \quad \lambda := \sqrt{\frac{B(2A'^3 - 9A')(3 - A^2)}{B'(2A^3 - 9A)(3 - A'^2)}}.$$

2.1.2 The group law

The set of points on a Montgomery curve, together with the point addition operation, forms an abelian group. Under this law, each point $P = (x, y)$ has inverse $-P = (x, -y)$, and the point at infinity 0_E acts as the identity element.

In what follows, for a point $P \neq 0_E$, we denote its coordinates by $P = (x_P, y_P)$. In practical implementations, it is common to use projective coordinates $(X : Y : Z)$, where $x = X/Z$ and $y = Y/Z$, in order to avoid costly field inversions in the formulas below. Moreover, we often rely on x -only arithmetic and represent points as $P = (X_P : Z_P)$, meaning that the sign of the y -coordinate is not tracked.

Point addition. Let $E_{A,B}/\mathbb{F}_q$ be a Montgomery curve, and let $P = (x_P, y_P)$ and $Q = (x_Q, y_Q)$ be points on $E_{A,B}$ such that $P \neq \pm Q$. Their sum $R = P + Q = (x_R, y_R)$ is computed as

$$x_R = B\lambda^2 - (x_P + x_Q) - A$$

and

$$y_R = \lambda(x_P - x_R) - y_P,$$

where $\lambda = (y_P - y_Q)/(x_P - x_Q)$. The point at infinity is the neutral element, so $P + 0_E = 0_E + P = P$.

Point doubling. Let $E_{A,B}/\mathbb{F}_q$ be a Montgomery curve, and let $P = (x_P, y_P)$ be a point on $E_{A,B}$ with $P \neq -P$. The doubling $[2]P = (x_{[2]P}, y_{[2]P})$ is given by

$$x_{[2]P} = \frac{(x_P^2 - 1)^2}{4x_P(x_P^2 + Ax_P + 1)},$$

and

$$y_{[2]P} = y_P \cdot \frac{(x_P^2 - 1)(x_P^4 + 2Ax_P^3 + 6x_P^2 + Ax_P + 1)}{8x_P^2(x_P^2 + Ax_P + 1)^2}.$$

If $P = -P$, then $[2]P = 0_E$.

Scalar multiplication. The group law naturally induces a scalar multiplication map $[k] : E \rightarrow E$ for $k \in \mathbb{Z}$. For $k > 0$, the value $[k]P$ is obtained by summing k copies of P , i.e., $[k]P = P + \dots + P$. For $k < 0$, we define $[k]P = -[|k|]P$, and for $k = 0$, we set $[0]P = 0_E$.

In practice, scalar multiplication is implemented using a sequence of point doublings and additions. The Montgomery ladder provides an efficient and regular method, ensuring that the number of operations grows logarithmically with k .

For a point $P \in E$, the smallest positive integer m such that $[m]P = 0_E$ is called the order of P .

Point difference. Given the x -coordinates x_P and x_Q of two points $P, Q \in E(\mathbb{F}_{p^2})$, one can deterministically compute the set $\{r_-, r_+\} = \{x_{P-Q}, x_{P+Q}\}$ via

$$r_{\pm} = \frac{B_{XZ} \pm \sqrt{B_{XZ}^2 - B_{ZZ}B_{XX}}}{B_{ZZ}},$$

where

$$\begin{aligned} B_{XX} &= (x_P x_Q - 1)^2, \\ B_{XZ} &= (x_P x_Q + 1)(x_P + x_Q) + 2Ax_P x_Q, \\ B_{ZZ} &= (x_P - x_Q)^2. \end{aligned}$$

Since $x_Q = x_{-Q}$, it is not possible to distinguish between x_{P-Q} and x_{P+Q} using x -coordinates alone. In practice, however, we are free to replace Q by $-Q$, and therefore it suffices to select one of the two values in a deterministic manner.

2.1.3 Torsion subgroups and deterministic basis computation

Let $m \in \mathbb{Z}$ and let $E/\mathbb{F}_{p^2}$ be a supersingular elliptic curve. We denote by $E[m]$ the m -torsion subgroup of E , that is, the set of all points $P \in E(\mathbb{F}_{p^2})$ such that $[m]P = 0_E$. If $m^2 \parallel \#E(\mathbb{F}_{p^2})$, then

$$E[m] \simeq \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/m\mathbb{Z},$$

and in particular $E[m]$ has cardinality m^2 .

Furthermore, there exist points $R, S \in E[m]$, not uniquely determined, such that $\langle R, S \rangle = E[m]$. We refer to such a pair (R, S) as a basis of $E[m]$.

For implementation purposes, it is convenient to represent a basis (R, S) using only x -coordinates, namely by the triplet (x_R, x_S, x_{RS}) . This representation determines the pair (R, S) only up to a global sign, meaning that the same triplet corresponds to both (R, S) and $(-R, -S)$.

2.2 Abelian varieties and Isogenies

For any abelian variety A , there exists a dual abelian variety $\hat{A}$ having the same dimension, and every isogeny $\varphi : A \rightarrow B$ admits a dual isogeny $\hat{\varphi} : \hat{B} \rightarrow \hat{A}$ of equal degree. A morphism $\lambda_A : A \rightarrow \hat{A}$ of this form is called a polarisation, and the pair (A, λ_A) is then referred to as a polarised abelian variety (PAV). In the special case where λ_A is an isomorphism, it is called a

principal polarisation, and A is said to be a principally polarised abelian variety (PPAV). Elliptic curves provide precisely the one-dimensional examples of PPAVs, since they carry a canonical principal polarisation.

2.2.1 Isogenies between elliptic curves

When $\dim(A) = 1$, the abelian variety A is an elliptic curve, which we denote by E . In this case, its dual abelian variety satisfies $\hat{E} = \text{Pic}^0(E) \simeq E$.

Let E_1 and E_2 be elliptic curves defined over $\mathbb{F}_q$. An isogeny between E_1 and E_2 is a non-constant morphism $\varphi : E_1 \rightarrow E_2$, given by rational functions in the coordinates with coefficients in $\mathbb{F}_q$, and satisfying $\varphi(0_{E_1}) = 0_{E_2}$. In particular, every isogeny is a group homomorphism from E_1 to E_2 . Moreover, the existence of an isogeny over $\mathbb{F}_q$ can be characterized in terms of point counts: the curves E_1 and E_2 admit an isogeny defined over $\mathbb{F}_q$ if and only if they have the same number of $\mathbb{F}_q$ -rational points, namely, if and only if $\#E_1(\mathbb{F}_q) = \#E_2(\mathbb{F}_q)$.

An isogeny is essentially determined by its kernel, namely the subgroup $\ker(\varphi) = \{P \in E_1 \mid \varphi(P) = 0_{E_2}\}$. More precisely, given a subgroup $G \subset E_1$ of cardinality N , there exists, up to post-composition with isomorphisms, a unique elliptic curve E_2 and an isogeny $\varphi : E_1 \rightarrow E_2$ of degree N such that $\ker(\varphi) = G$. Consequently, if $G = \langle Q \rangle$ is cyclic, the isogeny can be represented by a single generator Q .

Moreover, every isogeny $\varphi : E_1 \rightarrow E_2$ admits a unique dual isogeny $\hat{\varphi} : E_2 \rightarrow E_1$ of the same degree N , such that the compositions $\hat{\varphi} \circ \varphi$ and $\varphi \circ \hat{\varphi}$ correspond to the scalar multiplication map $[N]$ on E_1 and E_2 , respectively.

For explicit computations, an isogeny φ defined over $\mathbb{F}_q$ can be written in the form $\varphi((x, y)) = (f(x), y \cdot g(x))$, where $f(x)$ and $g(x)$ are rational functions over $\mathbb{F}_q$. Writing $f(x) = f_1(x)/f_2(x)$ as a ratio of coprime polynomials, the degree of the isogeny satisfies $\deg(\varphi) = \max\{\deg(f_1), \deg(f_2)\}$. Let $E/\mathbb{F}_q$ an elliptic curve in Montgomery form, then the separable isogeny can be expressed in the form $\varphi : (x, y) \mapsto (f(x), cyf'(x))$ where $c \in \mathbb{F}_q^*$.

2-isogenies Let $Q \in E_{A,B}$ be a point of order 2 generating the kernel of a 2-isogeny $\varphi : E_{A,B} \rightarrow E_{A',B'}$, which means that $\ker(\varphi) = \{Q, 0_{E_1}\}$. We must distinguish the following two cases:

- $Q = (0, 0)$: the isogeny is

$$\varphi(x, y) = \left(\frac{1}{\sqrt{A^2 - 4}} \cdot \frac{x^2 + Ax + 1}{x}, \frac{1}{\sqrt[4]{A^2 - 4}} \cdot y \cdot \frac{x^2 - 1}{x^2} \right),$$

and the image curve is defined by

$$(A', B') = \left(-\frac{2A}{\sqrt{A^2 - 4}}, B \right).$$

- $Q = (x_Q, 0)$ with $x_Q \neq 0$: in this case, we have

$$\varphi(x, y) = \left(\frac{x_Q x^2 - x}{x - x_Q}, \sqrt{x_Q} \cdot y \cdot \frac{x_Q x^2 - 2x_Q^2 x + x_Q}{(x - x_Q)^2} \right),$$

and

$$(A', B') = (2(1 - 2x_Q^2), B).$$

In both cases, the point $(0, 0)$ on $E_{A', B'}$ is in the kernel of the dual isogeny $\hat{\varphi}$. This ensures that the first formula is only used at the start of a (non-backtracking) chain of isogenies.

2.2.2 Isogenies between abelian varieties

In SQIsignTriangle, dimension-2 isogenies serve as an efficient mechanism for computing isogenies of non-smooth degree between elliptic curves. In this section, we present the basic framework for carrying out such computations in the setting of abelian varieties.

Principally polarized abelian surfaces (PPAS) arise as the two-dimensional analogue of elliptic curves. Concretely, they are algebraic varieties described by polynomial equations and endowed with a group structure whose law is expressed by rational functions. Over an algebraically closed field, every PPAS is isomorphic to one of the following two types:

1. a product of elliptic curves $E_1 \times E_2$;
2. the Jacobian $\text{Jac}(C)$ of a genus-2 hyperelliptic curve C .

The group law on the product abelian variety $A := E_1 \times E_2$ is defined componentwise from the group laws on the elliptic curves E_1 and E_2 . More precisely, for points $(P_1, P_2), (Q_1, Q_2) \in A$, their sum is given by

$$(P_1, P_2) \oplus (Q_1, Q_2) := (P_1 \oplus_1 Q_1, P_2 \oplus_2 Q_2),$$

where $\oplus_1$ and $\oplus_2$ denote the group operations on E_1 and E_2 , respectively, as described in Section 2.1.2. The identity element of this group structure is $0_A := (0_{E_1}, 0_{E_2})$.

An isogeny $\Phi : A_1 \rightarrow A_2$ between principally polarized abelian surfaces is a surjective morphism, given in coordinates by rational functions, that is compatible with the group structures on A_1 and A_2 . In the situations considered here, such isogenies fall into three classes:

- when $A_1 = E_1 \times E_2$ and $A_2 = \text{Jac}(C)$ for a genus-2 hyperelliptic curve C , the map Φ is called a gluing isogeny;
- when $A_1 = \text{Jac}(C)$ and $A_2 = E_1 \times E_2$, the map Φ is called a splitting isogeny;
- when A_1 and A_2 are Jacobians, Φ is referred to as a generic isogeny.

Similarly to isogenies between elliptic curves, isogenies between PPAS have a finite kernel $\ker(\Phi) = \{P \in A_1 \mid \Phi(P) = 0_{A_2}\}$ and are determined by their kernel up to post-composition with an isomorphism. We say that Φ is an (N, N) -isogeny when its kernel $\ker(\Phi)$ is generated by two linearly independent points $P, Q \in A_1$ of order N . We write $\ker(\Phi) = \langle P \rangle \oplus \langle Q \rangle$. Not every pair of linearly independent N -torsion points P, Q defines an isogeny. 'Principally polarized' of PPAS means that P and Q are *isotropic*, that is, they have trivial N -Weil pairing, namely $e_N(P, Q) = 1$.

Define that N factors as $\prod \ell_i^{e_i}$. Instead of evaluating an (N, N) -isogeny Φ directly at cost $O(N^2)$, we write it as

$$\Phi = \Phi_r \circ \cdots \circ \Phi_1,$$

where each Φ_i is an (ℓ_i, ℓ_i) -isogeny. This reduces the cost of computing Φ to $O(\sum e_i \ell_i^2)$. In SQIsignTriangle, we focus on the special case of $(2^e, 2^e)$ -isogenies

$$\Phi : E_1 \times E_2 \longrightarrow E_3 \times E_4$$

between products of elliptic curves defined over $\mathbb{F}_{p^2}$, and decompose such a map into a chain of e successive $(2, 2)$ -isogenies. For prime p in SQIsignTriangle, the first morphism in this chain, $\Phi_1 : E_1 \times E_2 \rightarrow A_1$, is expected to be a gluing isogeny, the final morphism $\Phi_e : A_{e-1} \rightarrow E_3 \times E_4$ a splitting isogeny, and all remaining intermediate morphisms generic.

Isogeny diamond. A (d_1, d_2) -isogeny diamond is a commutative diagram of isogenies between elliptic curves consisting of morphisms $\varphi_1 : E_0 \rightarrow E_1$, $\varphi_2 : E_0 \rightarrow E_2$, $\varphi'_1 : E_2 \rightarrow E_3$, and $\varphi'_2 : E_1 \rightarrow E_3$, where φ_1 and φ'_1 are d_1 -isogenies, and φ_2 and φ'_2 are d_2 -isogenies. These maps satisfy the commutativity condition $\varphi'_2 \circ \varphi_1 = \varphi'_1 \circ \varphi_2$.

$$\begin{array}{ccc} E_0 & \xrightarrow{\varphi_1} & E_1 \\ \varphi_2 \downarrow & & \downarrow \varphi'_2 \\ E_2 & \xrightarrow{\varphi'_1} & E_3 \end{array}$$

Figure 2: A (d_1, d_2) -isogeny diamond

If d_1 and d_2 are coprime, such a diagram can equivalently be interpreted as a pushforward square determined by φ_1 and φ_2 , or as a pullback square determined by φ'_1 and φ'_2 .

Kani's lemma. Given a (d_1, d_2) -isogeny diamond with $\gcd(d_1, d_2) = 1$, the isogeny $\Phi : E_0 \times E_3 \rightarrow E_1 \times E_2$ defined by

$$\Phi = \begin{pmatrix} \varphi_1 & \widehat{\varphi'_2} \\ -\varphi_2 & \widehat{\varphi'_1} \end{pmatrix}$$

is a $(d_1 + d_2, d_1 + d_2)$ -isogeny between these products of elliptic curves, endowed with their natural principal polarisations. Its kernel is $\ker \Phi = \{ ([d_1]P, \varphi'_2 \circ \varphi_1(P)) \mid P \in E_0[d_1 + d_2] \}$.

Computing a $(2, 2)$ -isogeny chain. We now describe how to compute a chain of $(2, 2)$ -isogenies. Let $\Phi : E_1 \times E_2 \longrightarrow E_3 \times E_4$ be a $(2^e, 2^e)$ -isogeny between products of elliptic curves. Given two generators P and Q of $\ker(\Phi)$, we explain how Φ can be evaluated by decomposing it into a sequence of $(2, 2)$ -isogenies

$$E_1 \times E_2 \xrightarrow{\Phi_1} A_1 \xrightarrow{\Phi_2} A_2 \cdots A_{e-2} \xrightarrow{\Phi_{e-1}} A_{e-1} \xrightarrow{\Phi_e} E_3 \times E_4.$$

To evaluate each isogeny in the chain, it is enough to determine the theta null point of its codomain, since this information suffices to evaluate the corresponding map. A straightforward approach for computing a chain of $(2, 2)$ -isogenies is the following:

1. Compute the gluing isogeny $\Phi_1 : E_1 \times E_2 \longrightarrow A_1$ from the 8-torsion points $[2^{e-3}]P$ and $[2^{e-3}]Q$ lying above $\ker(\Phi_1) = \langle [2^{e-1}]P, [2^{e-1}]Q \rangle$ using [GluingCodomain](#), and then evaluate $\Phi_1(P)$ and $\Phi_1(Q)$ with [GluingEval](#).
2. For each $i \in \{2, \dots, e-2\}$, compute the generic isogeny $\Phi_i : A_{i-1} \longrightarrow A_i$ from the 8-torsion points $[2^{e-i-2}](\Phi_{i-1} \circ \dots \circ \Phi_1(P))$ and $[2^{e-i-2}](\Phi_{i-1} \circ \dots \circ \Phi_1(Q))$ lying above $\ker(\Phi_i)$, using [GenericCodomainWith8Torsion](#).
3. Compute the generic isogeny $\Phi_{e-1} : A_{e-2} \longrightarrow A_{e-1}$ from the 4-torsion points $\Phi_{e-2} \circ \dots \circ \Phi_1(P)$ and $\Phi_{e-2} \circ \dots \circ \Phi_1(Q)$ lying above $\ker(\Phi_{e-1})$, using [GenericCodomainWith4Torsion](#).
4. Compute the generic isogeny $\Phi_e : A_{e-1} \longrightarrow E_3 \times E_4$ from the 2-torsion kernel points $\Phi_{e-1} \circ \dots \circ \Phi_1(P)$ and $\Phi_{e-1} \circ \dots \circ \Phi_1(Q)$, using [GenericCodomain](#).
5. Compute the coordinate change from the theta coordinate system on $E_3 \times E_4$ naturally induced by Φ_e to a product theta coordinate system, so that image points on $E_3 \times E_4$ can be expressed in $(X : Z)$ -Montgomery coordinates on each factor E_3 and E_4 , using [SplittingIsomorphism](#).

To avoid the square root computations appearing in Steps (3) and (4), we may instead begin with isotropic points $P, Q \in (E_1 \times E_2)[2^{e+2}]$ lying above $\ker(\Phi)$, such that $\ker(\Phi) = \langle [4]P, [4]Q \rangle$ and $e_{2^{e+2}}(P, Q) = 1$. Then the isogeny $\Phi_i : A_{i-1} \longrightarrow A_i, i \in \{2, \dots, e\}$ can be computed from the 8-torsion basis $\ker(\Phi_i) = \langle [2^{e-i}](\Phi_{i-1} \circ \dots \circ \Phi_1(P)), [2^{e-i}](\Phi_{i-1} \circ \dots \circ \Phi_1(Q)) \rangle$ using [GenericCodomainWith8Torsion](#). This refinement is only available when the 2^{e+2} -torsion points are defined over $\mathbb{F}_{p^2}$, which is not guaranteed in general.

2.3 Quaternions and ideals

Quaternion algebras generalize Hamilton's quaternions. They are 4-dimensional vector spaces over a base field, generated by $\{1, i, j, k\}$, and form non-commutative algebras satisfying $i^2 = a, j^2 = b$ and $ij = -ji = k$, for some a, b in the base field. Up to isomorphism, we consider the quaternion algebra ramified at p and ∞ , denoted by $B_{p,\infty}$. Since $p \equiv 3 \pmod{4}$, we fix a basis such that $i^2 = -1, j^2 = -p$, and $ij = -ji = k$.

An element of $B_{p,\infty}$ is represented as a 5-tuple of integers $(a, b, c, d, r) \in \mathbb{Z}^5$, corresponding to the rational quaternion

$$\frac{a + bi + cj + dk}{r},$$

where a canonical representation is obtained by reducing the common denominator.

We work both with individual elements $\alpha \in B_{p,\infty}$ and with full-rank lattices of the form $\alpha_1\mathbb{Z} + \alpha_2\mathbb{Z} + \alpha_3\mathbb{Z} + \alpha_4\mathbb{Z}$, where $(\alpha_1, \dots, \alpha_4)$ is a $\mathbb{Q}$ -basis of $B_{p,\infty}$. Arithmetic operations (addition, multiplication, etc.) follow the usual algebraic rules, with multiplication determined by $i^2 = -1, j^2 = -p$, and $ij = -ji = k$, and denominators reduced when necessary.

For $\alpha = \frac{a+bi+cj+dk}{r}$, the conjugate is defined as

$$\bar{\alpha} = \frac{a - bi - cj - dk}{r}.$$

The reduced trace and reduced norm are given by $\text{tr}(\alpha) = \alpha + \bar{\alpha}$ and $\text{nrd}(\alpha) = \alpha\bar{\alpha}$. The bilinear pairing $\langle \alpha, \beta \rangle = \text{tr}(\alpha\bar{\beta})$ defines a symmetric form on $B_{p,\infty}$, and satisfies $\langle \alpha, \alpha \rangle = 2 \text{nrd}(\alpha)$, showing that this quadratic space is positive definite.

2.3.1 Quaternion orders and ideals

An *order* is a lattice in $B_{p,\infty}$ that is also a subring. Elements of an order $\mathcal{O}$ are called integral, as they have reduced norm and trace in $\mathbb{Z}$. An order is said to be *maximal* if it is not properly contained in any larger order.

The quaternion algebra used in SQIsignTriangle contains a maximal order $\mathcal{O}_0$ with basis $(1, i, \frac{i+j}{2}, \frac{1+ij}{2})$, which in turn contains a (non-maximal) suborder with basis $(1, i, j, ij)$.

Let $\mathcal{O}$ be an order. A left (resp. right) *integral ideal* of $\mathcal{O}$ is a sublattice of $\mathcal{O}$ that is closed under left (resp. right) multiplication by $\mathcal{O}$. The left order of an ideal I is defined as

$$\mathcal{O}_L(I) = \{\alpha \in B_{p,\infty} \mid \alpha I \subseteq I\},$$

and similarly for the right order $\mathcal{O}_R(I)$.

The norm of an ideal I , denoted $\text{nrd}(I)$, is the greatest common divisor of the norms of its elements; it is an integer satisfying $\text{nrd}(I) = \sqrt{[\mathcal{O}_L(I) : I]} = \sqrt{[\mathcal{O}_R(I) : I]}$. Any ideal admits a representation of the form

$$I = \mathcal{O}_L(I)\alpha + \mathcal{O}_L(I) \text{nrd}(I)$$

for some $\alpha \in \mathcal{O}_L(I)$. We simplify notation by writing $\mathcal{O}\langle \alpha, N \rangle$ for $\mathcal{O}\alpha + \mathcal{O}N$.

The product IJ of two ideals satisfying $\mathcal{O}_R(I) = \mathcal{O}_L(J)$ is defined as the lattice product. It is again an ideal with $\mathcal{O}_L(IJ) = \mathcal{O}_L(I)$ and $\mathcal{O}_R(IJ) = \mathcal{O}_R(J)$, and the norm is multiplicative under this operation.

We define equivalence of orders via conjugation: $\mathcal{O}_1$ and $\mathcal{O}_2$ are equivalent if there exists $\beta \in B_{p,\infty}^*$ such that $\beta\mathcal{O}_1 = \mathcal{O}_2\beta$. Similarly, two left $\mathcal{O}$ -ideals I and J are equivalent if $I = J\beta$ for some $\beta \in B_{p,\infty}^*$. In this case, their right orders are also equivalent since $\beta\mathcal{O}_R(I) = \mathcal{O}_R(J)\beta$.

2.3.2 Basic operations on ideals.

Ideals are represented via their underlying lattices, and standard operations such as equality testing, membership, sum, intersection, and product are handled accordingly. For instance, to compute $\mathcal{O}\langle \alpha, N \rangle$, one first computes $\mathcal{O}\alpha$ and $\mathcal{O}N$ via lattice multiplication, and then takes their sum. Conversely, given an ideal I , one can find $\alpha \in I$ such that $I = \mathcal{O}\langle \alpha, N \rangle$ by sampling elements until one satisfies $\text{gcd}(\text{nrd}(\alpha), N^2) = N$.

Ideal inverse: All ideals considered here are connecting maximal orders and thus admit inverses. The inverse of such an ideal I is

$$I^{-1} = \frac{1}{\text{nrd}(I)} \bar{I},$$

where $\bar{I}$ is the lattice of conjugates of elements of I . Although I^{-1} is not itself an ideal, it is a rank-4 lattice satisfying $II^{-1} = \mathcal{O}_L(I)$ and $I^{-1}I = \mathcal{O}_R(I)$.

Left and right orders: These can be recovered from the inverse via $\mathcal{O}_L(I) = II^{-1}$ and $\mathcal{O}_R(I) = I^{-1}I$.

Connecting ideals: Given two orders $\mathcal{O}_L$ and $\mathcal{O}_R$, a connecting ideal can be constructed as $I = N\mathcal{O}_L\mathcal{O}_R$, where N is the square root of the index of $\mathcal{O}_L \cap \mathcal{O}_R$ in $\mathcal{O}_L$, or equivalently a common denominator of $\mathcal{O}_L\mathcal{O}_R$.

Pullback and pushforward: Let I, J be ideals with $\mathcal{O}_R(J) = \mathcal{O}_L(I)$ and coprime norms. The pullback is defined as

$$[J]^*I = JI + \text{nrd}(I)\mathcal{O}_L(J),$$

while the pushforward, for two left $\mathcal{O}$ -ideals I, J of coprime norm, is

$$[J]_*I = J^{-1}(J \cap I).$$

These satisfy the identity $[J]^*([J]_*I) = I$.

2.4 Converting between ideals and isogenies

In this section, we describe the correspondence between ideals and isogenies and how to move between these two representations.

2.4.1 Deuring Correspondence.

Let E be an elliptic curve. An *endomorphism* of E is an isogeny $\varphi : E \rightarrow E$, and the set of all such maps forms the *endomorphism ring* $\text{End}(E)$. When E is a supersingular elliptic curve defined over $\mathbb{F}_{p^2}$, the ring $\text{End}(E)$ is isomorphic to a maximal order $\mathcal{O}$ in the quaternion algebra $B_{p,\infty}$.

Fixing an isomorphism $\mathcal{O} \cong \text{End}(E)$, each element $\alpha \in \mathcal{O}$ corresponds to an endomorphism of E . By abuse of notation, we write $\alpha(P)$ for the action of this endomorphism on $P \in E$, and $\ker \alpha$ for its kernel.

This identification yields a bijection between left ideals of $\mathcal{O}$ of norm coprime to p and finite subgroups of E , which correspond to kernels of separable isogenies. More explicitly, an ideal I is associated to the subgroup

$$E[I] := \{P \in E \mid \alpha(P) = 0_E, \forall \alpha \in I\},$$

which, when $I = \mathcal{O}\langle\alpha, N\rangle$, reduces to $E[I] = \ker \alpha \cap E[N]$.

To each such ideal I , we associate a separable isogeny φ_I with kernel $E[I]$. Conversely, given a separable isogeny φ from E , the corresponding ideal is

$$I_\varphi = \{\alpha \in \mathcal{O} \mid \alpha(P) = 0_E, \forall P \in \ker \varphi\}.$$

In `SQIsignTriangle`, we restrict to primes $p \equiv 3 \pmod{4}$. In this case, the curve $E_0 : y^2 =$

$x^3 + x$ is supersingular, and its endomorphism ring is isomorphic to

$$\mathcal{O}_0 = \mathbb{Z} \oplus i\mathbb{Z} \oplus \frac{i+j}{2}\mathbb{Z} \oplus \frac{1+k}{2}\mathbb{Z}.$$

Under this isomorphism, j corresponds to the Frobenius endomorphism $(x, y) \mapsto (x^p, y^p)$, while i corresponds to the automorphism $(x, y) \mapsto (-x, \sqrt{-1}y)$ on E_0 .

Throughout this work, we fix this choice of E_0 , $\mathcal{O}_0$, and the isomorphism $\text{End}(E_0) \cong \mathcal{O}_0$. Recall that every supersingular elliptic curve E admits a model defined over $\mathbb{F}_{p^2}$, and in particular its j -invariant satisfies $j(E) \in \mathbb{F}_{p^2}$. The notation $\mathcal{O}$ denotes an arbitrary maximal order. Given an isogeny $\psi : E_0 \rightarrow E$ and its associated $(\mathcal{O}_0, \mathcal{O})$ -ideal I , the identification $\mathcal{O}_0 \cong \text{End}(E_0)$ naturally induces an isomorphism $\mathcal{O} \cong \text{End}(E)$.

Table 4: Deuring Correspondence

Supersingular elliptic curves	Quaternions
$j(E)$ or $j(E)^p$ supersingular	$\mathcal{O} \simeq \text{End}(E)$ maximal order in $B_{p,\infty}$
$\varphi : E \rightarrow E'$	left $\mathcal{O}$ -ideal and right $\mathcal{O}'$ -ideal I_φ
$\varphi, \psi : E \rightarrow E'$	$I_\varphi \sim I_\psi$ ($I_\psi = I_\varphi \alpha$)
$\hat{\varphi} : E' \rightarrow E$	$\overline{I_\varphi}$
$\varphi \circ \psi$	$I_\psi \cdot I_\varphi$
$\theta \in \text{End}(E)$	principal ideal $\mathcal{O}\theta$
$\deg(\varphi)$	$\text{nrd}(I_\varphi)$

2.4.2 Qlapoti: converting $\mathcal{O}_0$ -ideals to isogenies.

Let I be a left $\mathcal{O}_0$ -ideal and let E_I denote the codomain of φ_I . Given I , we describe how to compute E_I together with the images of P_0, Q_0 under φ_I ; the full procedure is summarized in [IdealToIsogeny](#).

Qlapoti [BSE⁺25] is a recent improvement of the ideal-to-isogeny translation procedure. Given a left $\mathcal{O}_0$ -ideal J , the goal is to construct two ideals I_1, I_2 equivalent to J such that

$$\text{nrd}(I_1) + \text{nrd}(I_2) = 2^e.$$

This condition allows one to apply the two-dimensional isogeny splitting technique based on Kani's lemma, and hence to recover the isogeny associated to J via the Deuring correspondence. Compared with previous approaches, which usually search for ideals satisfying a relaxed equation $u \text{nrd}(I_1) + v \text{nrd}(I_2) = 2^e$ and therefore require additional auxiliary isogenies of degrees u and v , Qlapoti directly solves the original norm equation $\text{nrd}(I_1) + \text{nrd}(I_2) = 2^e$. As a result, the ideal-to-isogeny translation can be performed using only one chain of $(2, 2)$ -isogenies, which significantly reduces both the running time and the memory consumption.

More precisely, Qlapoti first replaces J by a smallest equivalent ideal I and writes

$$I = \mathcal{O}_0 \langle n, \alpha \rangle, \quad n = \text{nrd}(I),$$

where $\alpha \in I$ is a small generator satisfying $\text{nrd}(\alpha) = nr$ with $\gcd(n, r) = 1$. Since every ideal

equivalent to I can be written in the form $\chi_I(\beta) = I\bar{\beta}/\text{nrd}(I)$ for some $\beta \in I$, Qlapoti searches for two elements $\beta_1, \beta_2 \in I$ such that

$$\text{nrd}(\beta_1) + \text{nrd}(\beta_2) = 2^e n.$$

The key simplification is to restrict the search to elements of the form

$$\beta_k = n(a_k + b_k i) + \alpha, \quad k = 1, 2,$$

where $a_k, b_k \in \mathbb{Z}$. Writing $\alpha = a_\alpha + b_\alpha i + c_\alpha j + d_\alpha k$, the above norm equation is transformed into

$$n(a_1^2 + b_1^2 + a_2^2 + b_2^2) + 2a_\alpha(a_1 + a_2) + 2b_\alpha(b_1 + b_2) = 2^e - 2r.$$

Reducing this equation modulo n , Qlapoti first finds a short solution (s, t) to the linear congruence

$$2a_\alpha s + 2b_\alpha t \equiv 2^e - 2r \pmod{n}.$$

Then it imposes the relations $a_1 + a_2 = s$ and $b_1 + b_2 = t$. This removes the linear terms and reduces the problem to a classical sum-of-two-squares equation

$$(2a_1 - s)^2 + (2b_1 - t)^2 = \frac{2(2^e - 2r - 2a_\alpha s - 2b_\alpha t)}{n} - s^2 - t^2.$$

This final equation can be solved efficiently using Cornacchia's algorithm. Once a solution is found, the algorithm recovers a_1, b_1, a_2, b_2 , constructs β_1, β_2 , and outputs

$$I_1 = \chi_I(\beta_1), \quad I_2 = \chi_I(\beta_2).$$

By construction, these two ideals are equivalent to J and satisfy $\text{nrd}(I_1) + \text{nrd}(I_2) = 2^e$, providing the required input for the subsequent two-dimensional IdealToIsogeny computation.

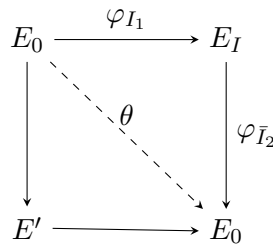


Figure 3: Qlapoti isogeny diamond

After applying $\text{Qlapoti}(J)$, we obtain two ideals I_1 and I_2 equivalent to J such that $\text{nrd}(I_1) + \text{nrd}(I_2) = 2^e$. Let $d_1 = \text{nrd}(I_1)$ and $d_2 = \text{nrd}(I_2)$. Since I_1 and I_2 lie in the same ideal class, their composition defines a principal ideal, and hence an endomorphism of E_0 . More precisely, under the fixed identification $\mathcal{O}_0 \simeq \text{End}(E_0)$, we obtain

$$\theta = \frac{\beta_2 \bar{\beta}_1}{n} = \varphi_{\bar{I}_2} \circ \varphi_{I_1} \in \mathcal{O}_0.$$

where $I_k = \chi_I(\beta_k)$ for $k = 1, 2$. The endomorphism θ has degree $d_1 d_2$, while $d_1 + d_2 = 2^e$. Let $E_0[2^e] = \langle P_0, Q_0 \rangle$. By evaluating θ on the basis points P_0 and Q_0 , we obtain the generators of the Kani kernel

$$\ker(\Phi) = \langle ([d_1]P_0, \theta(P_0)), ([d_1]Q_0, \theta(Q_0)) \rangle = \{([d_1]P, \theta(P)) \mid P \in E_0[2^e]\}.$$

This kernel defines a two-dimensional isogeny $\Phi : E_0 \times E_0 \longrightarrow E_I \times E'$, which can be computed as a chain of $(2, 2)$ -isogenies. Projecting the evaluation of Φ onto the first component then yields an efficient representation of the one-dimensional isogeny $\varphi_{I_1} : E_0 \longrightarrow E_I$. Since I_1 is equivalent to the original input ideal J , the codomain E_I is isomorphic to the desired codomain associated with J .

Since $(\bar{\beta}_1) = [n]I^{-1}I_1 = \bar{I}I_1$, $(\bar{\beta}_2) = [n]I^{-1}I_2 = \bar{I}I_2$, we have $\beta_1 = \hat{\varphi}_{I_1} \circ \varphi_I$, $\beta_2 = \hat{\varphi}_{I_2} \circ \varphi_I$, and then get $\varphi_I = [1/\text{nrd}(I_k)]\varphi_{I_k} \circ \beta_k$ for $k = 1, 2$. Finally, the images of P_0, Q_0 under φ_I can be obtained as

$$\varphi_I(P_0) = [1/\text{nrd}(I_1)]\varphi_{I_1} \circ \beta_1(P_0), \quad \varphi_I(Q_0) = [1/\text{nrd}(I_1)]\varphi_{I_1} \circ \beta_1(Q_0).$$

After expressing β_1 as a linear combination of the chosen basis of the corresponding maximal order, the matrix M_{β_1} is obtained by taking the same linear combination of the four precomputed basis matrices. In the algorithm, M_{β_1} is the matrix in $M_2(\mathbb{Z}/2^e\mathbb{Z})$ representing the action of β_1 on $E_I[2^e]$ with respect to the basis (P_I, Q_I) .

3 Algorithms Description

SQIsignTriangle relies on standard multi-precision arithmetic libraries, such as GMP, for handling large integers arising from system parameters and intermediate computations. Several low-level routines for finite-field arithmetic, lattice reduction, quaternion arithmetic, and theta-based isogeny evaluation are standard and are used as black-box subroutines in our implementation. We list these auxiliary routines below without giving their full specifications. Their standard implementations follow the corresponding routines used in SQIsign [AAA⁺25].

The following auxiliary routines are used in the algorithms described in the previous subsections:

- **BasicArithmetic**: takes as input integers or finite-field elements, and outputs the result of addition, subtraction, multiplication, inversion, division, or exponentiation.
- **UniformSampling**: takes as input a finite interval or finite set, and outputs a uniformly sampled element.
- **ModularArithmetic**: takes as input integers modulo n , and outputs the result of modular addition, multiplication, inversion, or reduction.
- **XGCD**: takes as input integers a, b , and outputs integers g, u, v such that $g = \gcd(a, b)$ and $ua + vb = g$.
- **Legendre**: takes as input an integer a and an odd prime p , and outputs the Legendre symbol of a modulo p .
- **PrimalityTest**: takes as input an integer n , and outputs whether n is probably prime or composite.
- **Factorization**: takes as input a positive integer n , and outputs its prime factorization.
- **SquareRoot**: takes as input a finite-field element a , and outputs a square root of a if it exists, and $\perp$ otherwise.
- **ModularSQRT**: takes as input an integer a and a prime modulus p , and outputs a square root of a modulo p if it exists, and $\perp$ otherwise.
- **HNF**: takes as input an integer matrix, and outputs its Hermite normal form.
- **LLL**: takes as input a lattice basis, and outputs an LLL-reduced basis of the same lattice.
- **BabaiRounding**: takes as input a reduced lattice basis B and a target vector v , and outputs a lattice vector close to v .
- **EndomorphismAction**: takes as input a quaternion element $\alpha \in \mathcal{O}_0$ and a torsion point $P \in E_0[2^f]$, and outputs $\alpha(P)$ under the fixed identification $\mathcal{O}_0 \simeq \text{End}(E_0)$.
- **WeilPairing**: takes as input two torsion points $P, Q \in E[n]$, and outputs their Weil pairing $e_n(P, Q)$.

- **DBL**: takes as input a point P in projective Montgomery coordinates and the curve coefficient, and outputs $[2]P$.
- **GluinCodomain**: takes as input kernel points defining a gluing $(2, 2)$ -isogeny, and outputs the codomain theta null point and auxiliary evaluation data.
- **GluinEvalSpecial**: takes as input a special product point of the form $(P, 0)$ or $(0, P)$ and gluing evaluation data, and outputs its image under the gluing isogeny.
- **GluinEval**: takes as input a general product point and gluing evaluation data, and outputs its image under the gluing isogeny.
- **ThetaPrecomp**: takes as input a theta null point, and outputs precomputed constants for theta arithmetic.
- **ThetaDBL**: takes as input a point in theta coordinates and precomputed theta constants, and outputs its double.
- **GenericCodomainWith8Torsion**: takes as input generic 8-torsion points T_1'', T_2'' on a domain surface A and get the kernel of $\Phi : A \rightarrow B$ as $\ker(\Phi) = \langle [4]T_1'' \rangle \oplus \langle [4]T_2'' \rangle$, and outputs the codomain theta null point and auxiliary evaluation data.
- **GenericCodomainWith4Torsion**: takes as input generic 4-torsion point T_1' on a domain surface A such that $[2]T_1' \in \ker(\Phi)$ and get the kernel data of $\Phi : A \rightarrow B$, and outputs the codomain theta null point and auxiliary evaluation data.
- **GenericCodomain**: given the theta null point of the domain, this routine computes the dual theta null point and its inverse, and the theta null point of the image under the generic $(2, 2)$ -isogeny $\Phi : A \rightarrow B$.
- **GenericEval**: takes as input a point in theta coordinates and generic evaluation data, and outputs its image under the corresponding $(2, 2)$ -isogeny.
- **SplittingIsomorphism**: takes as input a theta null point representing a product surface, and outputs an isomorphism to product form.
- **ThetaToProduct**: takes as input a theta null point in product form, and outputs the Montgomery coefficients of the two elliptic-curve factors.
- **ThetaProductPointToMontgomery**: takes as input a point in theta coordinates on a product surface, and outputs the corresponding point in Montgomery coordinates on $E_1 \times E_2$.

The constants used in the quaternion computations and in the ideal-to-isogeny translation are listed in Table 5. The values are chosen consistently with the SQIsign parameter choices for the corresponding security levels.

Parameter settings. Table 5 lists the implementation parameters used in our algorithms. The parameter `QUAT_primality_num_iter` specifies the number of Miller–Rabin iterations for primality tests and is not an explicit input to the algorithms.

Table 5: Implementation parameters used in our algorithms.

Parameter	Security level λ				Description
	128	160	256	512	
<code>QUAT_primality_num_iter</code>	32	32	32	32	Miller–Rabin iterations.
<code>QUAT_equiv_bound_coeff</code>	64	64	64	64	Coefficient bound in equivalent-ideal search.
<code>QUAT_prime_cofactor</code>	$2^{251} + 65$	$2^{313} + 125$	$2^{505} + 51$	$2^{1023} + 1155$	Smallest prime larger than $2^{\lceil \log_2(p) \rceil}$.

3.1 Elliptic curve arithmetic

Below we describe the algorithms for x -only arithmetic on Montgomery elliptic curves used throughout our implementation. These algorithms provide efficient primitives for scalar multiplication, point arithmetic, torsion basis generation, and basis conversion. The following routines are used:

- `xDBLADD` takes as input projective x -coordinates of P , Q , and $P - Q$, and outputs the projective x -coordinates of $[2]P$ and $P + Q$.
- `Ladder` takes as input a projective x -coordinate P and a scalar m , and outputs the projective x -coordinate of $[m]P$ using the Montgomery ladder.
- `ProjectiveDifference` takes as input projective x -coordinates of P and Q , and outputs a deterministic projective x -coordinate corresponding to either $P - Q$ or $P + Q$.
- `TorsionBasisToHint` takes as input a Montgomery curve E_A and an integer e , and deterministically outputs an x -only basis of $E_A[2^e]$ together with auxiliary hints.
- `TorsionBasisFromHint` takes as input a Montgomery curve E_A , an integer e , and the auxiliary hints, and reconstructs an x -only basis of $E_A[2^e]$.
- `ChangeOfBasis` takes as input two torsion bases and outputs the corresponding change-of-basis matrix.

Algorithm 3.1 xDBLADD($P, Q, P - Q, (A_{24} : C_{24})$) [AAA⁺25]

Input: Projective x -coordinates $P = (X_P : Z_P)$, $Q = (X_Q : Z_Q)$, $P - Q = (X_{P-Q} : Z_{P-Q})$, and the Montgomery coefficient $(A_{24} : C_{24})$ of the curve E .

Output: Projective x -coordinates $[2]P = (X_{2P} : Z_{2P})$ and $P + Q = (X_{P+Q} : Z_{P+Q})$.

```
1:  $t_0 \leftarrow X_P + Z_P$ 
2:  $t_1 \leftarrow X_P - Z_P$ 
3:  $X_{2P} \leftarrow t_0^2$ 
4:  $t_2 \leftarrow X_Q - Z_Q$ 
5:  $X_{P+Q} \leftarrow X_Q + Z_Q$ 
6:  $t_0 \leftarrow t_0 \cdot t_2$ 
7:  $Z_{2P} \leftarrow t_1^2$ 
8:  $t_1 \leftarrow t_1 \cdot X_{P+Q}$ 
9:  $t_2 \leftarrow X_{2P} - Z_{2P}$ 
10:  $Z_{2P} \leftarrow Z_{2P} \cdot C_{24}$ 
11:  $X_{2P} \leftarrow X_{2P} \cdot Z_{2P}$ 
12:  $X_{P+Q} \leftarrow A_{24} \cdot t_2$ 
13:  $Z_{P+Q} \leftarrow t_0 - t_1$ 
14:  $Z_{2P} \leftarrow Z_{2P} + X_{P+Q}$ 
15:  $X_{P+Q} \leftarrow t_0 + t_1$ 
16:  $Z_{2P} \leftarrow Z_{2P} \cdot t_2$ 
17:  $Z_{P+Q} \leftarrow Z_{P+Q}^2$ 
18:  $X_{P+Q} \leftarrow X_{P+Q}^2$ 
19:  $Z_{P+Q} \leftarrow Z_{P+Q} \cdot X_{P-Q}$ 
20:  $X_{P+Q} \leftarrow X_{P+Q} \cdot Z_{P-Q}$ 
21: return  $((X_{2P} : Z_{2P}), (X_{P+Q} : Z_{P+Q}))$ 
```

Algorithm 3.2 Ladder($P, (A_{24} : C_{24}), m$) [AAA⁺25]

Input: A projective x -coordinate $P = (X_P : Z_P)$, the Montgomery coefficient $(A_{24} : C_{24})$ of the curve E , and a positive scalar m with binary representation $m = (m_{k-1}, \dots, m_0)_2$.

Output: The projective x -coordinate $[m]P = (X_{[m]P} : Z_{[m]P})$.

```
1:  $((X_0 : Z_0), (X_1 : Z_1)) \leftarrow ((1 : 0), (X_P : Z_P))$ 
2: for  $i$  from  $k - 1$  down to 0 do
3:   if  $m_i = 1$  then
4:      $((X_1 : Z_1), (X_0 : Z_0)) \leftarrow \text{xDBLADD}((X_1 : Z_1), (X_0 : Z_0), (X_P : Z_P), (A_{24} : C_{24}))$ 
5:   else
6:      $((X_0 : Z_0), (X_1 : Z_1)) \leftarrow \text{xDBLADD}((X_0 : Z_0), (X_1 : Z_1), (X_P : Z_P), (A_{24} : C_{24}))$ 
7: return  $(X_0 : Z_0)$ 
```

Algorithm 3.3 ProjectiveDifference($P, Q, (A : C)$) [AAA⁺25]

Input: Projective x -coordinates $P = (X_P : Z_P)$ and $Q = (X_Q : Z_Q)$, and the Montgomery coefficient $(A : C)$.

Output: A deterministic projective x -coordinate x_{PQ} , corresponding to either $P - Q$ or $P + Q$.

- 1: $B_{XX} \leftarrow C \cdot (X_P X_Q - Z_P Z_Q)^2$
 - 2: $B_{XZ} \leftarrow C \cdot (X_P X_Q + Z_P Z_Q)(X_P Z_Q + Z_P X_Q) + 2A X_P X_Q Z_P Z_Q$
 - 3: $B_{ZZ} \leftarrow C \cdot (X_P Z_Q - Z_P X_Q)^2$
 - 4: $\gamma \leftarrow C \cdot (C \cdot Z_P Z_Q)^2$
 - 5: $B_{XX} \leftarrow \gamma \cdot B_{XX}$
 - 6: $B_{XZ} \leftarrow \gamma \cdot B_{XZ}$
 - 7: $B_{ZZ} \leftarrow \gamma \cdot B_{ZZ}$
 - 8: $\delta \leftarrow \text{SquareRoot}(B_{XZ}^2 - B_{XX} B_{ZZ})$
 - 9: $X_{PQ} \leftarrow \delta + B_{XZ}$
 - 10: $Z_{PQ} \leftarrow B_{ZZ}$
 - 11: **return** $x_{PQ} = (X_{PQ} : Z_{PQ})$
-

Algorithm 3.4 TorsionBasisToHint($E_A, 2^e$) [AAA⁺25]

Input: A Montgomery curve $E_A : y^2 = x^3 + Ax^2 + x$, and an integer $e \leq f$.

Output: An x -only basis (x_R, x_S, x_{RS}) of $E_A[2^e]$, together with two hints h_A, h .

- 1: Let A be the Montgomery coefficient of E_A .
 - 2: **if** A is a square **then**
 - 3: $h_A \leftarrow 1$
 - 4: **else**
 - 5: $h_A \leftarrow 0$
 - 6: $h \leftarrow 0$
 - 7: **if** $h_A = 1$ **then**
 - 8: **repeat**
 - 9: $h \leftarrow h + 1$
 - 10: $x_R \leftarrow -A/(1 + i \cdot h)$
 - 11: **until** $1 + h^2$ is not a square **and** $x_R \in E_A(\mathbb{F}_{p^2})$
 - 12: **else**
 - 13: **repeat**
 - 14: $h \leftarrow h + 1$
 - 15: $x_R \leftarrow h \cdot A$
 - 16: **until** $x_R \in E_A(\mathbb{F}_{p^2})$
 - 17: $x_{RS} \leftarrow -x_R - A$
 - 18: $x_R \leftarrow \text{Ladder}((x_R : 1), (A + 2 : 4), (p + 1)/2^e)$
 - 19: $x_{RS} \leftarrow \text{Ladder}((x_{RS} : 1), (A + 2 : 4), (p + 1)/2^e)$
 - 20: $x_S \leftarrow \text{ProjectiveDifference}(x_R, x_{RS}, (A : 1))$
 - 21: **if** $h \geq 128$ **then**
 - 22: $h \leftarrow 0$
 - 23: **return** (x_R, x_S, x_{RS}) and (h_A, h)
-

Algorithm 3.5 TorsionBasisFromHint($E_A, 2^e, h$) [AAA⁺25]

Input: A Montgomery curve $E_A : y^2 = x^3 + Ax^2 + x$, an integer $e \leq f$, together with a hint $h = (h_A, h_{\text{val}})$.

Output: An x -only basis (x_R, x_S, x_{RS}) of $E_A[2^e]$.

```
1: Let  $A$  be the Montgomery coefficient of  $E_A$ .
2: if  $h_{\text{val}} = 0$  then
3:    $(x_R, x_S, x_{RS}), (h_A, h_{\text{val}}) \leftarrow \text{TorsionBasisToHint}(E_A, 2^e)$ 
4:   return  $(x_R, x_S, x_{RS})$ 
5: else
6:   if  $h_A = 1$  then
7:      $x_R \leftarrow -A/(1 + i \cdot h_{\text{val}})$ 
8:   else
9:      $x_R \leftarrow h_{\text{val}} \cdot A$ 
10:  $x_{RS} \leftarrow -x_R - A$ 
11:  $x_R \leftarrow \text{Ladder}((x_R : 1), (A + 2 : 4), (p + 1)/2^e)$ 
12:  $x_{RS} \leftarrow \text{Ladder}((x_{RS} : 1), (A + 2 : 4), (p + 1)/2^e)$ 
13:  $x_S \leftarrow \text{ProjectiveDifference}(x_R, x_{RS}, (A : 1))$ 
14: return  $(x_R, x_S, x_{RS})$ 
```

Algorithm 3.6 ChangeOfBasis($E, (P_1, P_2), (Q_1, Q_2)$) [AAA⁺25]

Input: A basis (P_1, P_2) of $E[2^f]$ and a basis (Q_1, Q_2) of $E[2^e]$, where $e \leq f$.

Output: A change-of-basis matrix (x_i) , with $1 \leq i \leq 4$, such that

$$Q_1 = [x_1]\tilde{P}_1 + [x_2]\tilde{P}_2, \quad Q_2 = [x_3]\tilde{P}_1 + [x_4]\tilde{P}_2,$$

where $\tilde{P}_1 = [2^{f-e}]P_1$ and $\tilde{P}_2 = [2^{f-e}]P_2$.

```
1:  $\tilde{P}_1 \leftarrow [2^{f-e}]P_1$ 
2:  $\tilde{P}_2 \leftarrow [2^{f-e}]P_2$ 
3:  $\zeta \leftarrow t_{2^e}(\tilde{P}_1, \tilde{P}_2)$ 
4:  $\zeta_1 \leftarrow t_{2^e}(Q_1, \tilde{P}_2)$ 
5:  $\zeta_2 \leftarrow 1/t_{2^e}(Q_1, \tilde{P}_1)$ 
6:  $\zeta_3 \leftarrow t_{2^e}(Q_2, \tilde{P}_2)$ 
7:  $\zeta_4 \leftarrow 1/t_{2^e}(Q_2, \tilde{P}_1)$ 
8: for  $i = 1$  to 4 do
9:    $x_i \leftarrow \log_{\zeta}(\zeta_i)$ 
10: return  $(x_1, x_2, x_3, x_4)$ 
```

3.2 Quaternions and ideals

Below we describe the algorithms for quaternion algebra computations and ideal manipulation. These routines are used to solve norm equations, construct elements of prescribed norm, sample left ideals, and replace ideals by equivalent ideals with controlled norm. The following algorithms

are used:

- **Cornacchia** takes as input integers q and m , and outputs a solution to $x^2 + qy^2 = m$ if such a solution exists.
- **GeneralizedRepresentInteger** takes as input an integer M , an auxiliary element ω , and an order $\mathcal{O}$, and outputs an element $\gamma \in \mathcal{O}$ of prescribed reduced norm.
- **IdealGenerator** takes as input an ideal I and outputs a primitive generator $\gamma \in I$.
- **RandomIdealGivenNorm** takes as input an integer N and outputs a random left $\mathcal{O}_0$ -ideal of norm N .
- **RandomEquivalentPrimeIdeal** takes as input an ideal I and outputs an equivalent ideal of small prime norm.
- **Qlapoti** takes as input a left $\mathcal{O}_0$ -ideal I , and outputs equivalent ideals I_1 and I_2 satisfying $\text{nrd}(I_1) + \text{nrd}(I_2) = 2^e$, together with auxiliary elements used to construct the Kani kernel.
- **EquivalentSpecialIdeal** takes as input an ideal I and outputs an equivalent ideal whose norm satisfies a prescribed congruence condition.

Algorithm 3.7 Cornacchia(q, m)

Input: $q, m \in \mathbb{Z}$ with $m > 1$ and $0 \leq q \leq m$.

Output: $x, y \in \mathbb{Z}$ such that $x^2 + qy^2 = m$ if such x, y exist, and $\perp$ otherwise.

```
1: if  $q = 0$  then
2:   if  $m$  is a square then
3:     return  $(\sqrt{m}, 0)$ 
4:   else
5:     return  $\perp$ 
6: Factorize  $m = \prod_{i=1}^t p_i^{e_i}$ , where each  $p_i$  is prime and  $e_i \geq 1$ .
7:  $(x, y) \leftarrow (1, 0)$   $\triangleright$  Invariant:  $x^2 + qy^2$  equals the product already processed.
8: for  $i = 1, \dots, t$  do
9:    $p \leftarrow p_i, e \leftarrow e_i$ 
10:  if  $p = 2$  then
11:    if  $q = 1$  then
12:       $(a, b) \leftarrow (1, 1)$ 
13:    else
14:      return  $\perp$ 
15:  else
16:    if  $-q$  is not a square modulo  $p$  then
17:      return  $\perp$ 
18:     $r \leftarrow \text{ModularSQRT}(-q \bmod p, p)$ 
19:     $s \leftarrow p$ 
20:     $(r, s) \leftarrow (s, r \bmod s)$ 
21:     $(r, s) \leftarrow (s, r \bmod s)$ 
22:    while  $s^2 < p \leq r^2$  do
23:       $(r, s) \leftarrow (s, r \bmod s)$ 
24:     $a \leftarrow s$ 
25:     $B \leftarrow p - a^2$ 
26:    if  $B < 0$  or  $q \nmid B$  then
27:      return  $\perp$ 
28:     $C \leftarrow B/q$ 
29:    if  $C$  is not a square then
30:      return  $\perp$ 
31:     $b \leftarrow \sqrt{C}$ 
32:    if  $a^2 + qb^2 \neq p$  then
33:      return  $\perp$ 
34:    for  $j = 1, \dots, e$  do
35:       $(x, y) \leftarrow (xa - qyb, xb + ya)$   $\triangleright (x^2 + qy^2) \leftarrow (x^2 + qy^2)(a^2 + qb^2).$ 
36: if  $x^2 + qy^2 = m$  then
37:   return  $(x, y)$ 
38: else
39:   return  $\perp$ 
```

Algorithm 3.8 GeneralizedRepresentInteger($M, \omega, \mathcal{O}, \text{isogenyCond}$) [AAA⁺25]

Input: $M \in \mathbb{Z}$ odd such that $M > p$.

Input: $\omega \in B_{p,\infty}$ such that $q = -\omega^2$ is a positive integer and $q \equiv 1 \pmod{4}$.

Input: $\mathcal{O}$ a special extremal maximal order containing the suborder $\mathbb{Z}[\omega] + \mathbb{Z}[\omega]j \subset \mathcal{O}$, with j from the standard basis of $B_{p,\infty}$.

Input: `isogenyCond` boolean flag only relevant for $-\omega^2 = 1$.

Output: $\gamma \in \mathcal{O}$ with $\text{nrd}(\gamma) = M$, or raises an exception.

```
1: Initialize  $q \leftarrow -\omega^2$ ,  $\text{counter} \leftarrow 0$ ,  $\text{bound} \leftarrow \left\lceil \frac{4M}{p\sqrt{q}} \right\rceil$ , and  $\text{found} \leftarrow \text{false}$ 
2: while (not found) and (counter < bound) do
3:   counter  $\leftarrow$  counter + 1
4:   Sample  $z$  uniformly from  $[1, \dots, m]$  for  $m = \left\lfloor \sqrt{\frac{4M}{p} - q} \right\rfloor$ 
5:   Sample  $t$  uniformly from  $[-m', \dots, m']$  for  $m' = \left\lfloor \sqrt{\frac{4M - pz^2}{qp}} \right\rfloor$ 
6:    $M' \leftarrow 4M - p(z^2 + qt^2)$ 
7:   if  $M'$  is prime then
8:     res  $\leftarrow$  Cornacchia( $q, M'$ )
9:     found  $\leftarrow$  (res  $\neq \perp$ )
10:    if found then
11:       $(x, y) \leftarrow$  res
12:    if found and isogenyCond and  $q = 1$  then
13:      if  $x \not\equiv t \pmod{2}$  then
14:        Swap  $x$  and  $y$ 
15:      found  $\leftarrow$  ( $x - t \equiv 2 \pmod{4}$ ) and ( $y - z \equiv 2 \pmod{4}$ )
16:    if found then
17:       $\gamma \leftarrow x + \omega y + jz + \omega jt$ 
18:      Set  $d$  to be the biggest scalar such that  $\gamma/d \in \mathcal{O}$ 
19:      found  $\leftarrow$  ( $d = 2$ )
20:    if found then
21:      return  $\gamma/d$ 
22: raise Exception: "GeneralizedRepresentInteger failed"
```

Algorithm 3.9 IdealGenerator(I) [AAA⁺25]

Input: I : an ideal.

Output: γ primitive in I such that its norm N_I and γ generate I as an ideal of its left or right order.

```
1:  $n \leftarrow 0$ 
2: while true do
3:    $n \leftarrow n + 1$ 
4:   for  $a$  from  $-n$  up to  $n$  do
5:     for  $b$  from  $-n + |a|$  up to  $n - |a|$  do
6:       for  $c$  from  $-n + |a| + |b|$  up to  $n - |a| - |b|$  do
7:          $d \leftarrow n - |a| - |b| - |c|$ 
8:         if  $\gcd(a, b, c, d) = 1$  then
9:            $\gamma \leftarrow$  the quaternion of coordinates  $a, b, c, d$  in the HNF basis of  $I$ 
10:           $q \leftarrow \text{nrd}(\gamma)/N_I$ 
11:          if  $\gcd(q, N_I) = 1$  then
12:            return  $\gamma$ 
```

Algorithm 3.10 RandomIdealGivenNorm(N , prime) [AAA⁺25]

Input: A positive integer N not divisible by p which is the norm of some left $\mathcal{O}_0$ -ideal.

Input: A boolean prime indicating whether N is prime.

Input: A fixed parameter $m = \text{QUAT_prime_cofactor}$ which is prime, approximately as large as p and larger than N but distinct from both.

Output: A random left $\mathcal{O}_0$ -ideal J' of norm N , or raise an exception.

```
1: found  $\leftarrow$  false
2: if prime then
3:   while not found do
4:      $g_1, g_2, g_3 \leftarrow$  independent uniformly random integers in  $[0, N - 1]$ 
5:      $\gamma \leftarrow g_1i + g_2j + g_3ij$ 
6:     found  $\leftarrow (1 = \text{Legendre}(-\text{nrd}(\gamma), N))$ 
7:     if found then
8:        $\gamma \leftarrow \gamma + \text{ModularSQRT}(-\text{nrd}(\gamma), N)$ 
9: else
10:   $\gamma \leftarrow \text{GeneralizedRepresentInteger}(mN, i, \mathcal{O}_0, \text{false})$  ▷ This might raise an exception
11: found  $\leftarrow$  false
12: while not found do
13:   $x, y, z, w \leftarrow$  uniformly randomly selected integers in  $[1, N]$ 
14:   $\beta \leftarrow x + yi + zj + wij$ 
15:  found  $\leftarrow (\gcd(\text{nrd}(\beta), N) = 1)$ 
16:  $J' \leftarrow$  the left  $\mathcal{O}_0$ -ideal generated by  $\gamma\beta$  and  $N$ 
17: return  $J'$ 
```

Algorithm 3.11 RandomEquivalentPrimeIdeal(I) [AAA⁺25]

Input: I , a left $\mathcal{O}$ -ideal.

Output: $J \sim I$ of small prime norm, or raise an exception if unsuccessful.

```
1: counter  $\leftarrow 0$ 
2:  $b \leftarrow \text{QUAT\_equiv\_bound\_coeff}$ 
3: Compute an LLL-reduced basis  $\alpha_1, \alpha_2, \alpha_3, \alpha_4$  of  $I$ 
4: while counter  $< (2b + 1)^4$  do
5:   counter  $\leftarrow$  counter + 1
6:   Sample  $c_1, c_2, c_3, c_4$  uniformly at random from  $[-b, \dots, b]$ 
7:    $\beta \leftarrow \sum_{i=1}^4 c_i \alpha_i$ 
8:    $J \leftarrow \lambda_I(\beta)$ 
9:   if  $\text{nrd}(J)$  is prime then
10:    return  $J$ 
11: raise Exception: “RandomEquivalentPrimeIdeal failed”
```

Algorithm 3.12 Qlapoti(I) [BSE⁺25]

Input: Left $\mathcal{O}_0$ -ideal I . $\triangleright$ If the norm of the input ideal is not sufficiently small, we first use classical lattice reduction techniques to find a short element in the ideal and replace it by an equivalent ideal of smaller norm.

Output: Equivalent ideals I_1 and I_2 with $\text{nrd}(I_1) + \text{nrd}(I_2) = 2^e$, together with auxiliary elements β_1, β_2 and integers d_1, d_2 .

- 1: $n \leftarrow \text{nrd}(I)$
- 2: $\alpha \leftarrow \text{IdealGenerator}(I)$ $\triangleright$ small generator $\alpha = a_\alpha + b_\alpha i + c_\alpha j + d_\alpha k$ for $a_\alpha, b_\alpha, c_\alpha, d_\alpha \in \frac{1}{2}\mathbb{Z}$
- 3: $r \leftarrow \text{nrd}(\alpha)/n$
- 4: **if** $\gcd(2a_\alpha, n) \neq 1$ and $\gcd(2b_\alpha, n) \neq 1$ **then**
- 5: **goto** line 3
- 6: Compute shortest (s, t) such that $2a_\alpha s + 2b_\alpha t \equiv 2^e - 2r \pmod{n}$
- 7: Set $z := 2(2^e - 2r - 2a_\alpha s - 2b_\alpha t)/n - s^2 - t^2$
- 8: **if** $z < 0$ **then**
- 9: **goto** line 3
- 10: **if** $z \equiv 0 \pmod{4}$ and not $s \equiv t \equiv 0 \pmod{2}$ **then**
- 11: **goto** line 3
- 12: **if** $z \equiv 1 \pmod{4}$ and not $s \not\equiv t \pmod{2}$ **then**
- 13: **goto** line 3
- 14: **if** $z \equiv 2 \pmod{4}$ and not $s \equiv t \equiv 1 \pmod{2}$ **then**
- 15: **goto** line 3
- 16: **if** $z \equiv 3 \pmod{4}$ **then**
- 17: **goto** line 3
- 18: $\text{sol} \leftarrow \text{Cornacchia}(1, z)$ $\triangleright$ May return $\perp$ if no solution is found
- 19: **if** $\text{sol} = \perp$ **then**
- 20: **goto** line 3
- 21: $z_0, z_1 \leftarrow \text{sol}$ $\triangleright z_0^2 + z_1^2 = z$
- 22: **if** $z_0 \not\equiv s \pmod{2}$ **then**
- 23: swap z_0, z_1
- 24: $a_1 \leftarrow (z_0 + s)/2$, $b_1 \leftarrow (z_1 + t)/2$
- 25: $a_2 \leftarrow s - a_1$, $b_2 \leftarrow t - b_1$
- 26: $\beta_1 \leftarrow n(a_1 + b_1 i) + \alpha$
- 27: $\beta_2 \leftarrow n(a_2 + b_2 i) + \alpha$
- 28: $I_1 \leftarrow \chi_I(\beta_1)$
- 29: $I_2 \leftarrow \chi_I(\beta_2)$
- 30: $d_1 \leftarrow \text{nrd}(I_1)$
- 31: $d_2 \leftarrow \text{nrd}(I_2)$
- 32: **return** $I_1, I_2, \beta_1, \beta_2, d_1, d_2$

Algorithm 3.13 EquivalentSpecialIdeal(I, c_1, c_2)

Require: A left $\mathcal{O}$ -ideal I , target modulus c_1 , and remainder c_2 .

Ensure: An equivalent left $\mathcal{O}$ -ideal $J \sim I$ with $\text{nrd}(J) \equiv c_2 \pmod{c_1}$ and small norm.

```
1: Extract a  $\mathbb{Z}$ -basis  $B_{\text{raw}} = (b_1, b_2, b_3, b_4)^T$  of  $I$ 
2: Compute Gram matrix  $\mathbf{M} \leftarrow \frac{1}{\text{nrd}(I)} B_{\text{raw}} D B_{\text{raw}}^T$  where  $D = \text{diag}(1, 1, p, p)$ 
3: Define  $Q(x) = x \mathbf{M} x^T$  //  $Q(x)$  evaluates the norm of the equivalent ideal
4: while True do // Step 1 and 2: Find valid  $(u, w)$  pair
5:    $u_{\text{free}} \xleftarrow{\$} (\mathbb{Z}/c_1\mathbb{Z})^3$  // Randomly sample 3 free coordinates
6:    $u \leftarrow$  roots of  $Q(u) \equiv c_2 \pmod{c_1}$  for the remaining coordinate
7:    $C \leftarrow u(\mathbf{M} + \mathbf{M}^T)$ 
8:   if  $\exists j \in \{1, 2, 3, 4\}$  such that  $\gcd(C_j, c_1) = 1$  then
9:      $w_j \leftarrow -C_j^{-1} \sum_{i \neq j} C_i w_i \pmod{c_1}$  // Eliminate  $w_j$ 
10:     $w_{\text{free}} \xleftarrow{\$} (\mathbb{Z}/c_1\mathbb{Z})^2$  // Randomly sample 2 free coordinates
11:     $w \leftarrow$  roots of  $Q(w) \equiv 0 \pmod{c_1}$  for the remaining coordinate
12:    if solution  $w$  exists then
13:      break
14: Construct basis  $B_{\text{sublat}}$  for sublattice  $\Lambda_w = \{v \in \mathbb{Z}^4 \mid v \equiv \lambda w \pmod{c_1}\}$ 
15:  $D_{\text{embed}} \leftarrow \text{diag}(1, 1, \lfloor \sqrt{p} \rfloor, \lfloor \sqrt{p} \rfloor)$ 
16:  $B_{\text{embed}} \leftarrow B_{\text{sublat}} B_{\text{raw}} D_{\text{embed}}$  // Step 3: Embed into standard Euclidean space
17:  $T_{\text{sublat}} \leftarrow \text{LLL}(B_{\text{embed}})$  // Standard LLL
18:  $B_{\text{red}} \leftarrow T_{\text{sublat}} B_{\text{sublat}}$  // Reduced coordinate basis
19:  $c \leftarrow \lfloor -u \cdot B_{\text{red}}^{-1} \rfloor \in \mathbb{Z}^4$  // Step 4: Exact Babai rounding
20:  $x \leftarrow u + c B_{\text{red}} \in \mathbb{Z}^4$ 
21:  $\alpha \leftarrow x B_{\text{raw}} \in I$ 
22:  $J \leftarrow I \bar{\alpha} / \text{nrd}(I)$ 
23: return  $J$ 
```

3.3 Isogenies

Below we describe the isogeny-related algorithms used in our implementation. The following routines are used:

- **Isogeny22Chain** computes a chain of $(2, 2)$ -isogenies from a Kani kernel and evaluates input points.
- **IdealTolsogeny** converts a left $\mathcal{O}_0$ -ideal into the corresponding isogeny data, including the codomain curve and the images of the fixed basis (P_0, Q_0) .

Algorithm 3.14 Isogeny22Chain(P, Q, pts) [AAA⁺25]

Input: Points $P, Q \in E_1 \times E_2$ of order 2^e , a positive integer e , and an array pts containing points on $E_1 \times E_2$.

Output: The codomain $E_3 \times E_4$ of the $(2, 2)$ -isogeny chain $\Phi = \Phi_e \circ \dots \circ \Phi_1$ such that $\ker(\Phi) = \langle P, Q \rangle$ with $\deg(\Phi_k) = 2$ for all $k = 1, \dots, e$, and the evaluated points $[\Phi(P) \mid P \in \text{pts}]$.

```
1: Initialize a list  $\text{strat\_pts} \leftarrow [(P, Q)]$ 
2: Initialize a list  $\text{orders} \leftarrow [e - 2]$ 
3:  $k \leftarrow 0$ 
4: while  $\text{orders}[k] \neq 1$  do
5:    $k \leftarrow k + 1$ 
6:   if  $\text{orders}[k - 1] \geq 16$  then
7:      $n \leftarrow \lfloor \text{orders}[k - 1] / 2 \rfloor$ 
8:   else
9:      $n \leftarrow \text{orders}[k - 1] - 1$ 
10:   $(R, S) \leftarrow \text{strat\_pts}[k - 1]$ 
11:  for  $i$  from 0 up to  $n - 1$  do
12:     $R \leftarrow \text{DBL}(R, A_1)$ 
13:     $S \leftarrow \text{DBL}(S, A_2)$ 
14:   $\text{strat\_pts}[k] \leftarrow (R, S)$ 
15:   $\text{orders}[k] \leftarrow \text{orders}[k - 1] - n$ 
16:  $\_, I, 0_A, J, N \leftarrow \text{GluingCodomain}(\text{strat\_pts}[k])$ 
17:  $\text{pts} \leftarrow [\text{GluingEvalSpecial}(P, I, N) \mid P \in \text{pts}]$ 
18: for  $i$  from 0 up to  $k - 1$  do
19:    $\text{strat\_pts}[i] \leftarrow \text{GluingEval}(\text{strat\_pts}[i], I, J, N)$ 
20:    $\text{orders}[i] \leftarrow \text{orders}[i] - 1$ 
21:  $k \leftarrow k - 1$ 
22:  $\text{consts} \leftarrow \text{ThetaPrecomp}(0_A)$ 
23:  $j \leftarrow 0$ 
24: while  $k \geq 0$  and  $\text{orders}[k] \neq 0$  do
25:    $j \leftarrow j + 1$ 
26:   while  $\text{orders}[k] \neq 1$  do
27:      $k \leftarrow k + 1$ 
28:      $n \leftarrow \lfloor \text{orders}[k - 1] / 2 \rfloor$ 
29:     for  $i$  from 0 up to  $n - 1$  do
30:        $\text{strat\_pts}[k] \leftarrow \text{ThetaDBL}(\text{strat\_pts}[k - 1], \text{consts})$ 
31:      $\text{orders}[k] \leftarrow \text{orders}[k - 1] - n$ 
32:    $0'_A, I, 0_A \leftarrow \text{GenericCodomainWith8Torsion}(\text{strat\_pts}[k])$ 
33:    $\text{pts} \leftarrow [\text{GenericEval}(P, I) \mid P \in \text{pts}]$ 
34:    $\text{consts} \leftarrow \text{ThetaPrecomp}(0_A)$ 
35:   for  $i$  from 0 up to  $k - 1$  do
36:      $\text{strat\_pts}[i] \leftarrow \text{GenericEval}(\text{strat\_pts}[i], I)$ 
37:      $\text{orders}[i] \leftarrow \text{orders}[i] - 1$ 
38:    $k \leftarrow k - 1$ 
39: if  $e \geq 3$  then
40:    $\text{strat\_pts}[0] \leftarrow \text{GenericEval}(\text{strat\_pts}[0], I)$ 
41:    $\_, I, 0_A \leftarrow \text{GenericCodomainWith4Torsion}(\text{strat\_pts}[0], 0_A)$ 
42:    $\text{pts} \leftarrow [\text{GenericEval}(P, I) \mid P \in \text{pts}]$ 
43:    $\_, I, 0_A \leftarrow \text{GenericCodomain}(0_A)$ 
44:    $\text{pts} \leftarrow [\text{GenericEval}(P, I) \mid P \in \text{pts}]$ 
45:    $M \leftarrow \text{SplittingIsomorphism}(0_A)$ 
46:    $0_A \leftarrow M \cdot 0_A$ 
47:    $\text{pts} \leftarrow [M \cdot P \mid P \in \text{eval\_pts}]$ 
48:    $(A_3 : C_3), (A_4 : C_4) \leftarrow \text{ThetaToProduct}(0_A)$ 
49:    $\text{pts} \leftarrow [\text{ThetaProductPointToMontgomery}(P, 0_A) \mid P \in \text{pts}]$ 
50: Let  $E_3, E_4$  be the elliptic curves defined by Montgomery coefficients  $(A_3 : C_3), (A_4 : C_4)$ .
51: return  $E_3 \times E_4, \text{pts}$ 
```

Algorithm 3.15 IdealTolsogeny(I)

Input: A left $\mathcal{O}_0$ -ideal I .

Output: The codomain E_I of φ_I , together with $\varphi_I(P_0)$ and $\varphi_I(Q_0)$, where (P_0, Q_0) is a fixed basis of $E_0[2^f]$.

- 1: $I_1, I_2, \beta_1, \beta_2, d_1, d_2 \leftarrow \text{Qlapoti}(I)$ $\triangleright I_1 \sim I, I_2 \sim I$, and $d_1 + d_2 = 2^e$.
 - 2: $\theta \leftarrow \beta_2 \overline{\beta_1} / \text{nrd}(I)$ $\triangleright \theta \in \mathcal{O}_0 \simeq \text{End}(E_0)$.
 - 3: $P_e \leftarrow [2^{f-e}]P_0, \quad Q_e \leftarrow [2^{f-e}]Q_0$ $\triangleright E_0[2^e] = \langle P_e, Q_e \rangle$.
 - 4: $P_\theta \leftarrow \theta(P_e), \quad Q_\theta \leftarrow \theta(Q_e)$
 - 5: $K_P \leftarrow ([d_1]P_e, P_\theta)$
 - 6: $K_Q \leftarrow ([d_1]Q_e, Q_\theta)$ $\triangleright K_P$ and K_Q generate the Kani kernel.
 - 7: $B_P \leftarrow \beta_1(P_0), \quad B_Q \leftarrow \beta_1(Q_0)$
 - 8: $(x_1, x_2, x_3, x_4) \leftarrow \text{ChangeOfBasis}(E_0, (P_0, Q_0), (B_P, B_Q))$
 - 9: $M_{\beta_1} \leftarrow \begin{pmatrix} x_1 & x_2 \\ x_3 & x_4 \end{pmatrix}$ $\triangleright M_{\beta_1}$ is the matrix of β_1 on $E_0[2^f]$ in the basis (P_0, Q_0) .
 - 10: $E \times E', [(P, P'), (Q, Q')] \leftarrow \text{Isogeny22Chain}(K_P, K_Q, [(P_0, 0_{E_0}), (Q_0, 0_{E_0})])$
 - 11: **if** $e_{2^f}(P, Q) = e_{2^f}(P_0, Q_0)^{d_1}$ **then**
 - 12: $E_I \leftarrow E, \quad P_I \leftarrow P, \quad Q_I \leftarrow Q$
 - 13: **else**
 - 14: $E_I \leftarrow E', \quad P_I \leftarrow P', \quad Q_I \leftarrow Q'$
 - 15: $[P_I, Q_I]^T \leftarrow [d_1^{-1}]M_{\beta_1}[P_I, Q_I]^T$ $\triangleright d_1^{-1}$ is taken modulo 2^f .
 - 16: **return** $E_I, (P_I, Q_I)$
-

4 The Signature Scheme

In this section, we specifically introduce the key generation, signing, and verification algorithms of the SQIsignTriangle signature scheme.

4.1 Parameters

The SQIsignTriangle algorithms work with some parameters and precomputed values and we treat these parameters as input by default. We show the parameters of SQIsignTriangle as follows:

- The security parameter is denoted as λ and we usually choose $\lambda = 128, 160, 256, 512$ for different security levels.
- The character of finite field is prime $p = c \cdot 2^f - 1$, where c is a small positive integer and $f \approx 2\lambda$.
- The prime D_{mix} is used in generating secret key and of size around 4λ .
- A hash function `hash`, defined as $\text{hash} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda \parallel \{0, 1\}^\lambda$.

To make the signature more clearly, we present the signature in the following figure.

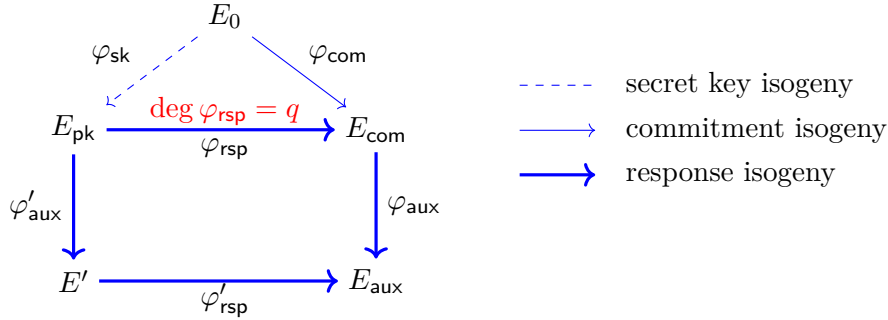


Figure 4: A description of response stage

4.2 Key generation

We present our key generation algorithm in Algorithm 4.1. The key generation algorithm of SQIsignTriangle produces a public key pk , including a supersingular elliptic curve E_{pk} , basis hint and a secret key sk including an ideal I_{sk} along with the torsion points information.

Specifically speaking, the key generation proceeds by sampling a random secret left $\mathcal{O}_0$ -ideal I_{sk} of prime norm D_{mix} by using `RandomIdealGivenNorm`. To translate the ideal I_{sk} into its corresponding isogeny, we use `RandomEquivalentPrimeIdeal` to obtain an ideal of smaller norm. Then the `IdealToIsogeny` will output the corresponding isogeny $\varphi_{\text{sk}} : E_0 \rightarrow E_{\text{pk}}$ along with the torsion points $\varphi_{\text{sk}}(P_0), \varphi_{\text{sk}}(Q_0)$. Since `RandomEquivalentPrimeIdeal` and `IdealToIsogeny` may fail, it might be necessary to try it with different input. To make the signature efficient, the information $(\varphi_{\text{sk}}(P_0), \varphi_{\text{sk}}(Q_0))$ is stored in the secret key sk . We generate a deterministic basis

$B_{\text{pk}} = (P_{\text{pk}}, Q_{\text{pk}})$ of $E_{\text{pk}}[2^f]$. Then we compute the change of basis matrix $\mathbf{M}_{\text{sk}}$ such that

$$M_{\text{sk}} \begin{pmatrix} \varphi_{\text{sk}}(P_0) \\ \varphi_{\text{sk}}(Q_0) \end{pmatrix} = \begin{pmatrix} P_{\text{pk}} \\ Q_{\text{pk}} \end{pmatrix}.$$

The matrix $\mathbf{M}_{\text{sk}}$ is part of the secret key sk . To find the torsion basis efficiently, we save a hint hint_{pk} in public key pk . In conclusion, the public key is $\text{pk} = (E_{\text{pk}}, \text{hint}_{\text{pk}})$ and the secret key $\text{sk} = (I_{\text{sk}}, \mathbf{M}_{\text{sk}})$

Algorithm 4.1 KeyGen(1^λ)

Input: The security parameter λ .

Output: Secret key sk and public key pk .

```

1: Initialize Found  $\leftarrow$  False
2: while True do
3:    $I_{\text{sk}} \leftarrow \text{RandomIdealGivenNorm}(D_{\text{mix}}, \text{True})$ 
4:   while not Found do
5:      $I_{\text{sk}} \leftarrow \text{RandomEquivalentPrimeIdeal}(I_{\text{sk}})$ 
6:     if  $I_{\text{sk}} \neq \text{False}$  then
7:       Found  $\leftarrow$  True
8:        $E_{\text{pk}}, \varphi_{\text{sk}}(P_0), \varphi_{\text{sk}}(Q_0) \leftarrow \text{IdealToIsogeny}(I_{\text{sk}})$ 
9:     else
10:      continue
11:    $P_{\text{pk}}, Q_{\text{pk}}, \text{hint}_{\text{pk}} \leftarrow \text{TorsionBasisToHint}(E_{\text{pk}}, 2^f)$ 
12:    $\mathbf{M}_{\text{sk}} \leftarrow \text{ChangeOfBasis}(E_{\text{pk}}, (\varphi_{\text{sk}}(P_0), \varphi_{\text{sk}}(Q_0)), (P_{\text{pk}}, Q_{\text{pk}}))$ 
13:    $\text{sk} \leftarrow (I_{\text{sk}}, \mathbf{M}_{\text{sk}})$ 
14:    $\text{pk} \leftarrow (E_{\text{pk}}, \text{hint}_{\text{pk}})$ 
15: return  $\text{sk}, \text{pk}$ 

```

4.3 Signing

We present our signing algorithm in Algorithm 4.2. Since our signature scheme is designed by constructing identification protocol via Fiat-Shamir technique, the signing algorithm is divided into three stages. Firstly, we compute the commitment isogeny $\varphi_{\text{com}} : E_0 \rightarrow E_{\text{com}}$. Secondly, we compute the hash value from the public key, message and commitment. Thirdly, we sign the message by using secret key, public key and the hash value. The signing procedure takes a secret key sk , pk and message msg as input and outputs a signature σ .

We first compute the commitment isogeny $\varphi_{\text{com}} : E_0 \rightarrow E_{\text{com}}$. This step is the same as key generation. We sample a random secret left $\mathcal{O}_0$ -ideal I_{com} of prime norm D_{mix} by using [RandomIdealGivenNorm](#). To translate the ideal I_{com} into its corresponding isogeny, we use [RandomEquivalentPrimeIdeal](#) to obtain an ideal of smaller norm. Then the [IdealToIsogeny](#) will output the corresponding isogeny $\varphi_{\text{com}} : E_0 \rightarrow E_{\text{com}}$ along with the torsion points $\varphi_{\text{com}}(P_0), \varphi_{\text{com}}(Q_0)$. Then we compute hash value $c_1, c_2 = \text{hash}(E_{\text{pk}} \parallel \text{msg} \parallel E_{\text{com}})$ via the public key, message and commitment. We compute the ideal $I_{\text{rsp}}^{(1)} = \overline{I_{\text{sk}}} \cdot I_{\text{com}}$. Rather than using this directly, we employ a specialized Euclidean embedding algorithm [EquivalentSpecialIdeal](#) to find an equivalent ideal

$I_{\text{rsp}} \sim I_{\text{rsp}}^{(1)}$ whose norm $q = n(I_{\text{rsp}})$ strictly satisfies the challenge constraint $q \equiv c_2 \pmod{c_1}$. To represent this isogeny efficiently, we use 2-dimensional isogeny representation

Algorithm 4.2 $\text{Sign}(\text{sk}, \text{pk}, \text{msg})$

Input: The secret key sk , the public key pk and the message msg

Output: The signature σ

- 1: Parse sk as $(I_{\text{sk}}, \mathbf{M}_{\text{sk}})$.
 - 2: Parse pk as $(E_{\text{pk}}, \text{hint}_{\text{pk}, 2^f}, P_{\text{pk}}, Q_{\text{pk}})$
 - 3: $c_1, c_2 \leftarrow \text{hash}(E_{\text{pk}} \parallel \text{msg} \parallel E_{\text{com}})$
 - 4: $I_{\text{rsp}}^{(1)} \leftarrow \overline{I_{\text{sk}}} \cdot I_{\text{com}}$
 - 5: $I_{\text{rsp}} \leftarrow \text{EquivalentSpecialIdeal}(I_{\text{rsp}}^{(1)}, c_1, c_2)$
 - 6: $q \leftarrow n(I_{\text{rsp}})$
 - 7: $e_{\text{rsp}} \leftarrow \lceil \log_2(q) \rceil$
 - 8: $I_{\text{aux}}^{(1)} \leftarrow \text{RandomIdealGivenNorm}(2^{e_{\text{rsp}}} - q, \text{False})$
 - 9: $I_{\text{aux}} \leftarrow (I_{\text{sk}} \cdot I_{\text{rsp}}) \cap I_{\text{aux}}^{(1)}$
 - 10: $(E_{\text{aux}}, P'_{\text{aux}}, Q'_{\text{aux}}) \leftarrow \text{IdealToIsogeny}(I_{\text{aux}})$
 - 11: $(P_{\text{aux}}, Q_{\text{aux}}) \leftarrow \mathbf{M}_{\text{sk}} \cdot (P'_{\text{aux}}, Q'_{\text{aux}})$
 - 12: **return** $\sigma \leftarrow (E_{\text{aux}}, P_{\text{aux}}, Q_{\text{aux}}, q)$
 - 13: **return** σ
-

4.4 Verification

We present the verification algorithm in Algorithm 4.3. On input a message msg , a signature $\sigma = (E_{\text{aux}}, P_{\text{aux}}, Q_{\text{aux}})$ and a public key $\text{pk} = (E_{\text{pk}}, \text{hint}_{\text{pk}})$, the verifier first checks the challenge compliance $q \equiv c_2 \pmod{c_1}$. To validate the correctness of the response isogeny φ_{rsp} without computing it directly, the verifier leverages Kani's lemma to embed the isogeny into a 2-dimensional Abelian variety.

The verifier reconstructs the torsion bases using the hints and scales the points by the cofactor $2^{f-e_{\text{rsp}}}$ to ensure they have exact order $2^{e_{\text{rsp}}}$. Using Kani's Lemma, the verifier constructs the $2^{e_{\text{rsp}}}$ -isogeny $\Phi : E_{\text{pk}} \times E_{\text{aux}} \rightarrow E_1 \times E_2$ with the 2-dimensional kernel $\{([q]P_{\text{pk}}, P_{\text{aux}}), ([q]Q_{\text{pk}}, Q_{\text{aux}})\}$. Finally, the verifier checks if the codomain splits into a product of elliptic curves where one of the components (E_1 or E_2) is isomorphic to the commitment E_{com} .

Algorithm 4.3 $\text{Verify}(\text{msg}, \text{pk}, \sigma)$

Input: A message msg , signature σ and public key pk .

Output: True if the signature is valid, False otherwise.

- 1: Parse pk as $(E_{\text{pk}}, \text{hint}_{\text{pk}})$
 - 2: Parse σ as $(E_{\text{aux}}, P_{\text{aux}}, Q_{\text{aux}}, q)$
 - 3: Check that E_{pk} is supersingular, otherwise **return** False .
 - 4: Check that E_{sig} is supersingular, otherwise **return** False .
 - 5: $P_{\text{pk}}, Q_{\text{pk}} \leftarrow \text{TorsionBasisFromHint}(E_{\text{pk}}, 2^f, \text{hint}_{\text{pk}})$
 - 6: $c_1, c_2 \leftarrow \text{hash}(E_{\text{pk}} \parallel \text{msg} \parallel E_{\text{com}})$
 - 7: Check that $q \equiv c_2 \pmod{c_1}$ is correct, otherwise **return** False .
 - 8: $e_{\text{rsp}} \leftarrow \lceil \log_2(q) \rceil$
 - 9: $\text{cff} \leftarrow 2^{f - e_{\text{rsp}}}$
 - 10: $P_{\text{pk}} \leftarrow [\text{cff}]P_{\text{pk}}, \quad Q_{\text{pk}} \leftarrow [\text{cff}]Q_{\text{pk}}$
 - 11: $P_{\text{aux}} \leftarrow [\text{cff}]P_{\text{aux}}, \quad Q_{\text{aux}} \leftarrow [\text{cff}]Q_{\text{aux}}$
 - 12: $F_1 \times F_2, _ \leftarrow \text{Isogeny22Chain}([q]P_{\text{pk}}, P_{\text{aux}}, ([q]Q_{\text{pk}}, Q_{\text{aux}}), [])$
 - 13: **if** $F_1 = E_{\text{com}}$ or $F_2 = E_{\text{com}}$ **then**
 - 14: **return** True
 - 15: **else**
 - 16: **return** False
-

5 Design Rationale

This chapter describe the design rationale of the signature scheme SQIsignTriangle.

5.1 Σ -protocols and Fiat-Shamir technique

In this section, we recall Σ -protocols, the Fiat–Shamir transformation, and the standard EUF-CMA security guarantee for Fiat–Shamir signatures in the random oracle model.

Definition 1 (Effective relation). An **effective relation** is a binary relation $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$, where $\mathcal{X}$, $\mathcal{Y}$, and $\mathcal{R}$ are efficiently recognizable sets. Elements of $\mathcal{Y}$ are called **statements**. If $(x, y) \in \mathcal{R}$, then x is called a **witness** for y .

Definition 2 (Σ -protocol). Let $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ be an effective relation. A Σ -protocol for $\mathcal{R}$ is a pair (P, V) of interactive algorithms with challenge space $\mathcal{C}$.

- The prover P takes as input a witness-statement pair $(x, y) \in \mathcal{R}$.
- The verifier V takes as input a statement $y \in \mathcal{Y}$ and outputs either accept or reject.
- The interaction consists of three messages followed by deterministic verification:
 1. P computes a commitment t and sends it to V .
 2. V samples a challenge $c \xleftarrow{\$} \mathcal{C}$ and sends it to P .
 3. P computes a response z and sends it to V .
 4. V accepts or rejects as a deterministic function of y and the conversation (t, c, z) .

The protocol satisfies **completeness** if, for every $(x, y) \in \mathcal{R}$, an honest interaction between $P(x, y)$ and $V(y)$ is accepted.

Definition 3 (Special soundness). Let (P, V) be a Σ -protocol for $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ with challenge space $\mathcal{C}$. We say that (P, V) has **special soundness** if there exists an efficient deterministic algorithm Ext such that, for any statement $y \in \mathcal{Y}$ and any two accepting conversations (t, c, z) and (t, c', z') for y with the same commitment t and distinct challenges $c \neq c'$, the algorithm Ext outputs a witness $x \in \mathcal{X}$ satisfying $(x, y) \in \mathcal{R}$.

Definition 4 (Special honest-verifier zero-knowledge). Let (P, V) be a Σ -protocol for $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ with challenge space $\mathcal{C}$. We say that (P, V) is **special honest-verifier zero-knowledge**, or **special HVZK**, if there exists an efficient probabilistic algorithm Sim such that:

1. for every $(y, c) \in \mathcal{Y} \times \mathcal{C}$, $\text{Sim}(y, c)$ outputs a pair (t, z) for which (t, c, z) is an accepting conversation for y ;
2. for every $(x, y) \in \mathcal{R}$, if $c \xleftarrow{\$} \mathcal{C}$ and $(t, z) \xleftarrow{\$} \text{Sim}(y, c)$, then the distribution of (t, c, z) is identical to the distribution of an honest conversation between $P(x, y)$ and $V(y)$.

The Fiat–Shamir transformation replaces the verifier’s random challenge in a public-coin identification protocol with the output of a hash function. Let (P, V) be a Σ -protocol for $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ with challenge space $\mathcal{C}$, and let $H : \{0, 1\}^* \rightarrow \mathcal{C}$ be a hash function modeled as a random oracle.

Definition 5 (Fiat–Shamir transformation). *Let (P, V) be a Σ -protocol for $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$. The Fiat–Shamir signature scheme $\Sigma_{\text{FS}} = (\text{Gen}, \text{Sign}, \text{Ver})$ is defined as follows.*

- **Gen**(1^λ): *Sample a witness-statement pair $(x, y) \in \mathcal{R}$ according to the key generation distribution. Output $\text{sk} = x$ and $\text{pk} = y$.*
- **Sign**(sk, m): *On input $\text{sk} = x$ and a message $m \in \{0, 1\}^*$, run the first move of P to compute a commitment t . Compute $c = H(y, t, m)$ and then compute the prover’s response z to challenge c . Output $\sigma = (t, z)$.*
- **Ver**(pk, m, σ): *On input $\text{pk} = y$, a message m , and a signature $\sigma = (t, z)$, compute $c = H(y, t, m)$. Accept if (t, c, z) is an accepting conversation for y , and reject otherwise.*

Theorem 1 (EUF-CMA security of Fiat–Shamir signatures). *Let (P, V) be a complete Σ -protocol for an effective relation $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$. Assume that (P, V) satisfies special soundness and special HVZK, and that the induced witness-search problem is hard for statements generated by **Gen**. Then, under the standard Fiat–Shamir security conditions in the random oracle model, the signature scheme obtained by applying the Fiat–Shamir transformation to (P, V) is existentially unforgeable under chosen-message attacks.*

The theorem is stated in the random oracle model. In particular, the Fiat–Shamir transformation does not in general imply standard-model security for an arbitrary concrete hash function.

5.2 Σ -protocol of SQIsignTriangle

GPS [GPS17] was the first signature scheme based on the hardness of computing the endomorphism ring of a supersingular elliptic curve. However, its underlying identification protocol suffers from a 1-bit challenge space, rendering it highly inefficient and impractical for real-world applications. Revisiting the 1-bit challenge limitation of the GPS scheme, a natural approach to expand the challenge space would be to directly require an isogeny of degree q between the public key curve E_{pk} and the commitment curve E_{com} . However, this immediately presents several technical hurdles. If q is chosen such that $q \approx 2^\lambda \approx \sqrt{p}$ (where λ is the security parameter and p is the field characteristic), an isogeny of exactly that degree may not even exist between E_{pk} and E_{com} . Relying on rejection sampling with multiple attempts yields an impractically low success probability. While it is known that an isogeny of a sufficient degree generally exists between any two supersingular elliptic curves [CFL⁺18], scaling the bit-length of q to $\log p \approx 2\lambda$ introduces a different obstacle: efficiently computing an isogeny of this specific size. This task reduces to finding a vector of a fixed length in a 4-dimensional lattice, a problem for which only exponential-time algorithms currently exist [BKM⁺24, AAF⁺26]. Alternatively, increasing the bit-length of q to $O((\log p)^{9/2})$ allows for the use of the generalized KLPT algorithm; however, this severely degrades performance, resulting in signing and verification times that are significantly slower than both SQIsign and PRISM.

To make the protocol run efficiently in practice, we propose a modular approach to the degree of challenges. Specifically, instead of requiring a fixed degree of response isogenies, we

slightly relax the constraint and require that the degree of response isogenies satisfies a modular equation, and the Σ -protocol runs as follows.

1. The prover generates the secret isogeny $\varphi_{\text{sk}} : E_0 \rightarrow E_{\text{pk}}$, the public key is $\text{pk} = E_{\text{pk}}$ and the secret key is $\text{sk} = \varphi_{\text{sk}}$.
2. The prover generates the isogeny $\varphi_{\text{com}} : E_0 \rightarrow E_{\text{com}}$ and the send E_{com} to the verifier E_{com} as the commitment.
3. A verifier selects two integers c_1, c_2 and send them to the prover.
4. The prover computes with a two-dimensional representation of an isogeny $\varphi_{\text{rsp}} : E_{\text{pk}} \rightarrow E_{\text{com}}$ of degree q such that $q \equiv c_2 \pmod{c_1}$.

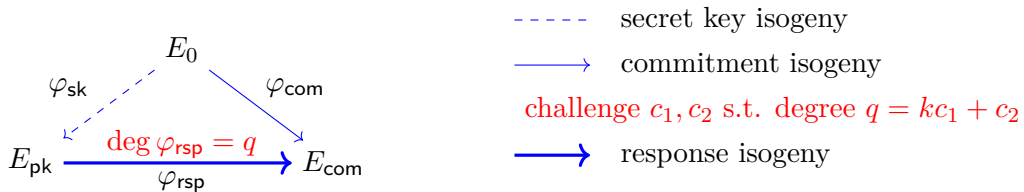


Figure 5: A description of the Σ -protocol

5.3 Hardness assumption and parameter choices

We give the hardness assumption of SQIsignTriangle.

Problem 1 (Endomorphism Ring Problem). *Given a supersingular elliptic curve E defined over K , find its endomorphism ring.*

Problem 2 (One Endomorphism Problem). *Given a supersingular elliptic curve E defined over K , find a non-scalar endomorphism $\alpha \in \text{End}(E)$.*

This is the hard language to prove the knowledge soundness of our Σ -protocol.

$$\{(E_{\text{pk}}, \alpha) \mid \alpha \in \text{End}(E_{\text{pk}}) \setminus \mathbb{Z} \text{ in efficient representation} \}$$

We will discuss the security of our signature in Section 6.

Parameter choices. Since we need digital signature schemes with classical security strengths of 128, 256, and 512 bits and quantum security strengths of 80, 128, and 256 bits, for SQIsignTriangle, the characteristic p of its finite field determines the overall security strength of the signature scheme. To meet the security conditions, we must satisfy

$$p^{1/2} > 2^{\lambda_1} \quad \text{and} \quad p^{1/4} > 2^{\lambda_2},$$

where λ_1, λ_2 denotes the required classical and quantum security strength. Therefore, the size of our finite field characteristic p is approximately $\max\{2^{2\lambda_1}, 2^{4\lambda_2}\}$, which gives the following result, for other concrete parameters, we introduce them in Section 7.

Table 6: SQIsignTriangle parameter sets

Classical security	Quantum security	Field prime p
128	64	$5 \cdot 2^{248} - 1$
160	80	$9 \cdot 2^{309} - 1$
256	128	$27 \cdot 2^{500} - 1$
512	256	$113 \cdot 2^{1016} - 1$

Concrete security analysis. The fastest known algorithms for Problem 1 have classical complexity in $\tilde{O}(p^{1/2})$ [DG16] and quantum complexity in $\tilde{O}(p^{1/4})$ using Grover’s algorithm. This problem is equivalent to the One Endomorphism Problem [PW24], which we stated as Problem 2. Therefore, for each parameter set, the claimed classical and quantum security levels are obtained by choosing the size of p so that the best known classical attack costs at least 2^λ operations and the best known quantum attack costs at least the corresponding quantum security target. Concretely, for the submitted instances with $\lambda \in \{160, 256, 512\}$, the parameter p is chosen so that $\tilde{O}(p^{1/2})$ is at least 2^{160} , 2^{256} , and 2^{512} , respectively, while $\tilde{O}(p^{1/4})$ is at least 2^{80} , 2^{128} , and 2^{256} , respectively. Thus, the concrete security analysis explicitly accounts for the best known classical and quantum attacks for each declared security level.

5.4 Failure Cases

We present possible failure cases of various algorithms of SQIsignTriangle. We adopt some estimates from SQIsign. We will guarantee [IdealGenerator](#) will successful find the result in a few number of steps.

The Failure Cases of Chains of $(2, 2)$ -isogenies

In the procedure of Key generation, signing, and verifying, $(2, 2)$ -isogenies are required. This procedure is usually divided into three parts including the first gluing isogeny $E_1 \times E_2 \rightarrow A_1$, the second generic isogeny $A_1 \rightarrow A_2 \dots \rightarrow A_{e-1}$, and the third split isogeny $A_e \rightarrow E_3 \times E_4$.

The computation of a chain of $(2, 2)$ -isogenies is deterministic once the kernel is fixed, so its failures should not be viewed as arising from the absence of a valid isogeny, but rather from exceptional geometric or algebraic degeneracies encountered along the chain. In particular, two types of failures may occur in an honest execution.

The first type of failure arises when an intermediate abelian surface in the chain splits prematurely as a product before the final step. In this case, the generic formulas used for the propagation of the chain no longer apply in the intended form. Under the standard heuristic that the intermediate surfaces encountered along the chain behave like uniformly random superspecial principally polarized abelian surfaces, the probability of hitting such a product locus is negligible; more precisely, since products of supersingular elliptic curves $E_1 \times E_2$ form a subset of size $O(p^2)$ inside a space of superspecial PPAS of size $O(p^3)$, this failure probability is of order $\tilde{O}(p^{-1})$. This fail only occurs in the procedure of Key generation and signing, but never happens in the procedure of verification of an honest signature.

The second type of failure occurs at the initial gluing step, when the auxiliary change-of-basis

matrix degenerates. This happens precisely when a certain trace quantity attached to the gluing kernel vanishes. Assuming, again heuristically, that this trace behaves like a uniformly random element of $\mathbb{F}_{p^2}$ during an honest execution, the probability of such a degeneracy is $O(p^{-2})$. Since only $O(1)$ two-dimensional isogeny computations are performed in each invocation of the procedure, the total contribution of this second failure source remains $O(p^{-2})$.

Combining the two estimates, the failure probability of [Isogeny22Chain](#) in an honest execution is negligible. In particular, the dominant contribution is the first one, so it is natural to summarize the overall failure probability of the $(2, 2)$ -isogeny chain by $\tilde{O}(p^{-1})$.

The Failure Cases of Ideal-to-isogeny translation

In key generation and signing, [IdealTolsogeny](#) is an important building block in `SQIsignTriangle`. It may fail in the following places:

1. [Qlapoti](#): If the call to [Qlapoti](#) fails.

In this ideal-to-isogeny translation, the only genuinely probabilistic step inside [Qlapoti](#) is the search for suitable generators in the norm-equation routine. Let I be the reduced ideal returned by the preprocessing step, and let $n = \text{nrd}(I)$. Following the heuristic analysis of [Qlapoti](#), the quotient $I/n\mathcal{O}$ may be identified with a non-trivial principal left ideal of $M_2(\mathbb{Z}/n\mathbb{Z})$, so the number of available generators is on the order of n^2 , more precisely bounded below by approximately $0.607927 n^2$. Imposing the norm constraint required by [Qlapoti](#) further restricts the lift α of such a generator, and under the standard heuristic that the relevant coefficients behave as random integers modulo n , the probability that a sampled generator is suitable is approximately $\frac{2^e}{4pn}$. Consequently, the expected number of suitable generators is about $0.607927 n \cdot \frac{2^e}{4p}$.

To translate this estimate into a failure probability, one models the successive trials of generators α as independent Bernoulli experiments with a common success probability. If c denotes the expected number of trials required for [Qlapoti](#) to terminate, then the success probability of a single trial is $q = 1/c$, and the probability of failure after k trials is $(1 - q)^k$. Requiring this quantity to be below $2^{-(\lambda+1)}$ yields the condition $k > (\lambda + 1) \log(2) c$. Comparing this with the above estimate for the number of suitable generators gives the threshold

$$L = (\lambda + 1) \cdot \frac{4 \log(2) p c}{0.607927 \cdot 2^e}.$$

Using the same heuristic as in the [Qlapoti](#) analysis, one then separates the failure event according to whether $n \leq L$ or $n > L$, obtaining

$$\Pr[\text{Qlapoti fails}] \leq \Pr[n \leq L] + \Pr[\text{Qlapoti fails} \mid n > L].$$

The first term is controlled by the distribution of small equivalent ideals, while the second is bounded by $2^{-(\lambda+1)}$. This leads to heuristic upper bounds 2^{-197} , 2^{-254} , 2^{-438} , and 2^{-944} for security levels $\lambda = 128, 160, 256$, and 512 , respectively. Therefore, within the complete [IdealTolsogeny](#) procedure, the contribution of the norm-equation step is already cryptographically negligible.

2. **Isogeny22Chain**: If the calls to **Isogeny22Chain** fail. As previously mentioned, the dominant contribution comes from the possibility that an intermediate abelian surface splits prematurely, which occurs with probability $\tilde{O}(p^{-1})$, while the degeneracy at the initial gluing step contributes only $O(p^{-2})$.

Overall, the failure probability of the complete **IdealTolsogeny** procedure can be bounded by the union bound

$$\Pr[\text{IdealTolsogeny fails}] \leq \Pr[\text{Qlapoti fails}] + \Pr[\text{Isogeny22Chain fails}].$$

Hence, in an honest execution, the overall failure probability is still negligible; more precisely, it is dominated by the $(2, 2)$ -isogeny-chain failure term, whereas the contribution of **Qlapoti** is already far below the target security level.

The Failure Cases of **RandomIdealGivenNorm**

RandomIdealGivenNorm is used to sample the secret, commitment, and auxiliary ideals. When the requested norm N is prime, the routine samples a trace-zero quaternion satisfying a quadratic-residuosity condition modulo N ; this succeeds after a geometric number of trials and does not call **GeneralizedRepresentInteger**. When N is composite, as in the auxiliary-ideal sampling step, the routine calls **GeneralizedRepresentInteger** and propagates its failure.

For the composite-norm case, **GeneralizedRepresentInteger** samples auxiliary integers and then tries to solve a binary norm equation using **Cornacchia**. Under the standard heuristic that the represented integers behave like random integers of comparable size with respect to primality and quadratic residuosity, the parameter `QUAT_repres_bound_input` is chosen so that the probability that all trials fail is below 2^{-64} . Thus **RandomIdealGivenNorm** contributes at most a 2^{-64} rejection term in the signing paths where composite norms are sampled.

The Failure Cases of **EquivalentSpecialIdeal**

The **EquivalentSpecialIdeal** algorithm represents the core algorithmic divergence of **SQIsignTriangle**. It bypasses the traditional, failure-prone prime-norm search by extracting the target degree q directly via lattice reduction. Consequently, its failure modes are purely geometric and algebraic. We rigorously analyze these failure probabilities below and mathematically demonstrate that the algorithm succeeds in expected polynomial time.

- **Norm Overflow via LLL Approximation**: The algorithm embeds the modular challenge $n(J) \equiv c_2 \pmod{c_1}$ into a 4-dimensional integer sublattice $\Lambda_w \subset \mathbb{Z}^4$ with volume $V = c_1^2 \cdot \frac{p}{4}$. To minimize the output degree q , it applies the LLL algorithm to extract a short vector $x \in \Lambda_w$. By the Gaussian heuristic, the shortest non-zero vector length $\lambda_1(\Lambda_w)$ is expected to be $Q(x) \approx \frac{\sqrt{2}}{\pi} V^{1/2} = \frac{\sqrt{2}}{\pi} c_1 p^{1/2}$. The LLL algorithm mathematically guarantees finding a vector x_{LLL} such that:

$$Q(x_{\text{LLL}}) \leq 2^{\frac{4-1}{2}} \cdot \lambda_1(\Lambda_w) \approx 2^{3/2} \frac{\sqrt{2}}{\pi} c_1 p^{1/2}.$$

The resulting isogeny degree is extracted as $q = Q(x_{\text{LLL}})/c_1 \leq O(p^{1/2})$. Because the LLL output is strictly bounded by this polynomial factor, the probability of a "norm overflow" (i.e., q exceeding the polynomial bounds required by the subsequent Qlapoti evaluation) is computationally negligible, yielding $\Pr[\text{overflow}] \approx 0$.

- **Imprimitivity and Cyclicity Failure:** For the subsequent Kani's lemma embedding to function correctly, the derived isogeny must be cyclic. Algebraically, this strictly requires the reconstructed quaternion element α (and thus the extracted lattice vector x) to be primitive (i.e., $\gcd(x_1, x_2, x_3, x_4) = 1$). The probability that a uniformly distributed random vector in a 4-dimensional integer lattice is primitive is asymptotically given by the inverse of the Riemann zeta function at $s = 4$:

$$P_{\text{prim}} = \frac{1}{\zeta(4)} = \frac{90}{\pi^4} \approx 0.9239.$$

Therefore, the probability that the extracted shortest vector is imprimitive, leading to a degenerate, non-cyclic ideal, is strictly bounded by a constant:

$$P_{\text{fail}} = 1 - P_{\text{prim}} \approx 0.0761 \text{ (or 7.61\%)}.$$

When this constant-probability failure occurs, the algorithm must discard x_{LLL} and restart the current step by selecting the next shortest vector in the reduced basis or applying a randomized unimodular transformation.

Expected Polynomial-Time Complexity: Because the LLL reduction guarantees $\Pr[\text{overflow}] \approx 0$, the success of a single iteration depends entirely on the primitivity of the extracted vector. Let T_{iter} be the execution time of one LLL reduction over Λ_w , which operates in strict polynomial time $O(\text{poly}(\log p))$.

The randomization loop terminates as soon as a valid, primitive vector is found. Since each lattice randomization provides an independent, identically distributed candidate vector, the number of iterations N follows a geometric distribution with success probability $P_{\text{succ}} \geq 1/\zeta(4)$. The expected number of iterations is strictly bounded by:

$$\mathbb{E}[N] = \frac{1}{P_{\text{succ}}} \leq \zeta(4) \approx 1.08.$$

Consequently, the expected total running time of the `EquivalentSpecialIdeal` algorithm is:

$$\mathbb{E}[T_{\text{total}}] = \mathbb{E}[N] \cdot T_{\text{iter}} = O(\text{poly}(\log p)).$$

This formally proves that, despite the occasional 7.61% imprimitivity failures, the algorithm efficiently recovers and outputs a valid equivalent ideal in expected polynomial time, ensuring the robust practical viability of the `SQIsignTriangle` signature generation.

The Impact of Failure Cases on Security

The protocol treats algorithmic failures by rejection. If a randomized subroutine fails during key generation or signing, the current attempt is discarded and the algorithm restarts with fresh

randomness. If a failure is encountered during verification, the verifier rejects the signature. Hence an algorithmic failure never provides an additional accepting path; it only changes the distribution by conditioning on successful termination.

This conditioning can have a small distributional effect. In key generation, some candidate ideals may be rejected more often than others, so the resulting public key distribution may be slightly biased. In signing, the same phenomenon can occur for the commitment curve and for the auxiliary isogeny used in the response. These biases do not change the underlying hard problem or the special-soundness argument, because failed attempts are not output as keys or transcripts. Their possible effect is only on indistinguishability: a real distribution conditioned on success may differ slightly from the ideal distribution used in the proof.

The size of this effect is bounded by the failure probability. If a sampling step fails with probability ε , then conditioning on success changes the distribution by statistical distance at most ε . For at most s signing queries, the total additional distinguishing loss is therefore at most $s\varepsilon$ by a standard hybrid argument. In verification, rejection is even less problematic for unforgeability, since a malicious signature that triggers a failure is rejected and therefore cannot help the adversary win the EUF-CMA game.

In `SQIsignTriangle`, ε is obtained by summing the relevant failure terms from the preceding paragraphs: the 2^{-64} bound for composite-norm calls to `RandomIdealGivenNorm`, the failure probability of `IdealToIsogeny`, the negligible $(2, 2)$ -isogeny-chain failure probability, and any residual abort probability if a finite retry bound is imposed on `EquivalentSpecialIdeal`. Since all these terms are negligible at the selected parameter levels, algorithmic failures add only negligible statistical losses to the security bounds.

6 Security Statements and Analyses

In this section, we state the assumptions used in the security proof and derive the EUF-CMA security of the Fiat–Shamir signature scheme obtained from the underlying Σ -protocol.

As highlighted by Aardal et al. [ABF⁺25], proving zero knowledge for signatures based on the Deuring correspondence is subtle. The honest response distribution depends on hidden algebraic information, so a simulator cannot efficiently reproduce it without additional input. We therefore use the *Fiat–Shamir with hints* paradigm, where the missing information is encoded as non-interactive hints. Following the level-structure viewpoint of Borin et al. [BFLS26], our Σ -protocol does not treat the challenge as an explicit challenge isogeny. This allows us to define the simulated hint directly through a response isogeny sampled from a prescribed admissible distribution. The resulting proof relies on two non-interactive assumptions: the hardness of the hinted one-endomorphism problem and the indistinguishability between the real response-isogeny distribution and the first isogeny component sampled from $\mathcal{H}_E^{\text{sim}}$.

6.1 Hint Distribution and Hardness Assumptions

We first define the simulated hint distribution used by the hint-assisted simulator. The distribution $\mathcal{H}_E^{\text{sim}}$ provides a response isogeny sampled from the prescribed admissible distribution, together with the auxiliary isogeny needed to format the simulated response.

Experiment 1: Hint distribution $\mathcal{H}_E^{\text{sim}}$

$\mathcal{H}_E^{\text{sim}}$:

1. Sample $c_1, c_2 \xleftarrow{\$} \{2^{\lambda/2}, 2^{\lambda/2} + 1, \dots, 2^{\lambda/2+1}\}$.
2. Sample an isogeny $\psi_1 : E \rightarrow E_1$ such that:
 - a. E_1 is sampled from the stationary distribution on the set of supersingular elliptic curves over $\mathbb{F}_{p^2}$;
 - b. the conditional distribution of ψ_1 given E_1 is uniform over admissible isogenies $\psi_1 : E \rightarrow E_1$ such that $d = \deg(\psi_1) < p^{7/8}$ and $d \equiv c_2 \pmod{c_1}$.
3. Sample an isogeny $\psi_2 : E_1 \rightarrow E_2$ of degree $2^a - d$, where $a = \lceil \log_2 d \rceil$.
4. Return $h = (c_1, c_2, \psi_1, \psi_2)$.

The first computational assumption is a hinted one-endomorphism problem. It captures the hardness of extracting a non-scalar endomorphism even in the presence of simulated hints.

Problem 3 (r -hint-OneEnd _{p}). *Given a supersingular elliptic curve E defined over $\mathbb{F}_{p^2}$ and r hints $h_1, \dots, h_r \leftarrow \mathcal{H}_E^{\text{sim}}$, find a non-scalar endomorphism in $\text{End}(E) \setminus \mathbb{Z}$ in efficient representation.*

For an algorithm $\mathcal{A}$, we denote its advantage in solving r -hint-OneEnd _{p} by

$$\text{Adv}^{r\text{-hint-OneEnd}_p}(\mathcal{A}) := \Pr[\alpha \leftarrow \mathcal{A}(E, h_1, \dots, h_r) : \alpha \in \text{End}(E) \setminus \mathbb{Z}],$$

where the hints are sampled as $h_1, \dots, h_r \leftarrow \mathcal{H}_E^{\text{sim}}$. We assume that r -hint-OneEnd $_p$ is hard for every $r = \text{poly}(\lambda)$; namely, for every PPT algorithm $\mathcal{A}$, $\text{Adv}^{r\text{-hint-OneEnd}_p}(\mathcal{A})$ is negligible.

The second assumption isolates the only distributional difference between the real and simulated response generation: the response isogeny itself.

Problem 4 (r -hint-dist). *Let $(\text{pk}, I_{\text{sk}}) \leftarrow \text{KeyGen}(1^\lambda)$ with $\text{pk} = E_{\text{pk}}$. Given pk and a tuple $(\phi_1, \dots, \phi_r)$ sampled with probability $1/2$ from one of the following two distributions:*

- $\mathcal{D}_0 = (\text{pk}, \phi_1, \dots, \phi_r)$, where for each $i \in \{1, \dots, r\}$, $I_{\text{com},i} \leftarrow \text{Commit}(\text{pk}, I_{\text{sk}})$, $\text{chl}_i = (c_{1,i}, c_{2,i}) \xleftarrow{\$} \mathcal{C}$, $I_{\text{rsp},i} \leftarrow \text{EquivalentSpecialIdeal}(\overline{I_{\text{sk}}} I_{\text{com},i}, c_{1,i}, c_{2,i})$, and $\phi_i = \varphi_{\text{rsp},i} : E_{\text{pk}} \rightarrow E_{\text{com},i}$ is the response isogeny induced by $I_{\text{rsp},i}$.
- $\mathcal{D}_1 = (\text{pk}, \phi_1, \dots, \phi_r)$, where for each $i \in \{1, \dots, r\}$, $h_i = (c_{1,i}, c_{2,i}, \psi_{1,i}, \psi_{2,i}) \xleftarrow{\$} \mathcal{H}_{E_{\text{pk}}}^{\text{sim}}$, and $\phi_i = \psi_{1,i}$.

The goal is to distinguish whether the tuple is sampled from $\mathcal{D}_0$ or $\mathcal{D}_1$.

For an algorithm $\mathcal{A}$, we denote its advantage on r -hint-dist by

$$\text{Adv}^{r\text{-hint-dist}}(\mathcal{A}) = |\Pr[\mathcal{A}(\text{pk}, x) = 1 \mid x \leftarrow \mathcal{D}_0] - \Pr[\mathcal{A}(\text{pk}, x) = 1 \mid x \leftarrow \mathcal{D}_1]|.$$

We assume that r -hint-dist is hard for every $r = \text{poly}(\lambda)$; namely, for every PPT algorithm $\mathcal{A}$, $\text{Adv}^{r\text{-hint-dist}}(\mathcal{A})$ is negligible.

The Fiat-Shamir-with-hints framework uses the following abstract hardness notion.

Definition 6 (Hard relation with hints). *Let $(R, \mathcal{H})$ be a relation with hints, where $\mathcal{H} = \{\mathcal{H}_x\}_x$ is a family of hint distributions indexed by statements. We say that $(R, \mathcal{H})$ is hard with hints if, for every $r = \text{poly}(\lambda)$ and every PPT algorithm $\mathcal{A}$, the following advantage is negligible:*

$$\text{Adv}_{(R, \mathcal{H})}^{\text{hint-rel}}(\mathcal{A}, r) := \Pr \left[(x, w^*) \in R \mid (x, w) \leftarrow \text{Gen}_R(1^\lambda), \right. \\ \left. h_1, \dots, h_r \leftarrow \mathcal{H}_x, \quad w^* \leftarrow \mathcal{A}(x, h_1, \dots, h_r) \right].$$

In our instantiation, the abstract hard-relation term for the one-endomorphism relation, equipped with $\mathcal{H}_E^{\text{sim}}$, is exactly the r -hint-OneEnd $_p$ advantage defined above.

6.2 Statistical and Knowledge Properties of the Σ -Protocol

We next record the statistical and extraction properties of the underlying Σ -protocol. The min-entropy bound controls the maximum point probability of honest commitments, while the extraction-error bound is used to derive the knowledge error of the protocol.

Definition 7 (α -min-entropy). *Let Σ be a Σ -protocol, and let com denote the commitment generated by the honest prover. We say that Σ has α -min-entropy if*

$$H_\infty(\text{com}) := -\log_2 \left(\max_{\text{com}^*} \Pr[\text{com} = \text{com}^*] \right) \geq \alpha.$$

Equivalently, every possible commitment occurs with probability at most $2^{-\alpha}$.

Lemma 1 (α -min-entropy of the Σ -protocol). *The underlying Σ -protocol satisfies the α -min-entropy property with $\alpha \geq \log_2 p - \log_2(15)$.*

Proof. Let μ be the distribution of the honest commitment curve E_{com} , and let π be the stationary distribution on the set of supersingular elliptic curves over $\mathbb{F}_{p^2}$. In the protocol, E_{com} is obtained as the endpoint of a random cyclic isogeny of prime degree D_{mix} , where D_{mix} is the smallest prime greater than $2^{4\lambda}$.

By the random-walk theorem for supersingular isogeny graphs, the point-wise deviation from the stationary distribution satisfies

$$\|\mu - \pi\|_\infty \leq \frac{2\sqrt{D_{\text{mix}}}}{D_{\text{mix}} + 1}.$$

Since $D_{\text{mix}} > 2^{4\lambda}$ and $p \approx 2^{2\lambda}$, we have $D_{\text{mix}} > p^2$, up to the constant-size discrepancy between p and $2^{2\lambda}$. Hence $2\sqrt{D_{\text{mix}}}/(D_{\text{mix}} + 1) < 2/p$. Moreover, by the Deuring mass formula, the stationary mass of any supersingular curve is at most $13/p$. Therefore, for every supersingular curve E^* ,

$$\Pr[E_{\text{com}} = E^*] \leq \pi(E^*) + \|\mu - \pi\|_\infty < \frac{13}{p} + \frac{2}{p} = \frac{15}{p}.$$

By the definition of min-entropy, this gives $H_\infty(E_{\text{com}}) \geq -\log_2(15/p) = \log_2 p - \log_2(15)$. Thus, the protocol satisfies the α -min-entropy property with $\alpha \geq \log_2 p - \log_2(15)$. In particular, when $p \approx 2^{2\lambda}$, this yields $\alpha \geq 2\lambda - \log_2(15)$ up to the constant-size discrepancy between p and $2^{2\lambda}$. $\square$

Lemma 2 (Soundness extraction error). *Given two valid transcripts $(E_{\text{com}}, \text{chl}, \sigma)$ and $(E_{\text{com}}, \text{chl}', \sigma')$ for independent challenges $(c_1, c_2) \neq (c'_1, c'_2)$, the probability that the canonically extracted endomorphism α is a scalar is bounded by $\Pr[\alpha \in \mathbb{Z}] \leq 2^{-7\lambda/4}$.*

Proof. The responses σ and σ' decode to cyclic isogenies $\varphi_{\text{rsp}}, \varphi'_{\text{rsp}} : E_{\text{pk}} \rightarrow E_{\text{com}}$ of degrees $d \equiv c_2 \pmod{c_1}$ and $d' \equiv c'_2 \pmod{c'_1}$. The extractor computes $\alpha = \widehat{\varphi'_{\text{rsp}}} \circ \varphi_{\text{rsp}} \in \text{End}(E_{\text{pk}})$. Suppose that the failure event occurs, namely $\alpha = [m] \in \mathbb{Z}$.

Taking degrees gives $dd' = m^2$. Since $[m] = \widehat{\varphi'_{\text{rsp}}} \circ \varphi_{\text{rsp}}$, the multiplication map $[m]$ factors through φ_{rsp} . Hence $\ker(\varphi_{\text{rsp}}) \subseteq \ker([m]) = E_{\text{pk}}[m]$. Since φ_{rsp} is cyclic of degree d , its kernel contains an element of order d . The group $E_{\text{pk}}[m]$ has exponent m , so $d \mid m$. Applying the same argument to $\widehat{\varphi_{\text{rsp}}} \circ \varphi'_{\text{rsp}} = [m]$ gives $d' \mid m$.

The conditions $d \mid m$ and $d' \mid m$, together with $dd' = m^2$, force $d = d' = m$. Substituting this back, we obtain $\widehat{\varphi'_{\text{rsp}}} \circ \varphi_{\text{rsp}} = [d]$. Since also $\widehat{\varphi_{\text{rsp}}} \circ \varphi_{\text{rsp}} = [d]$, we get $\widehat{\varphi'_{\text{rsp}}} \circ \varphi_{\text{rsp}} = \widehat{\varphi_{\text{rsp}}} \circ \varphi_{\text{rsp}}$. As φ_{rsp} is surjective, cancellation gives $\widehat{\varphi'_{\text{rsp}}} = \widehat{\varphi_{\text{rsp}}}$, and hence $\varphi'_{\text{rsp}} = \varphi_{\text{rsp}}$. Therefore, scalar extraction failure can occur only if the two response degrees collide.

It remains to bound the probability of $d = d'$. Write $d = kc_1 + c_2$ and $d' = k'c'_1 + c'_2$. Let $C = 2^{\lambda/2}$ and $K = 2^{5\lambda/4}$. The variables c_1, c_2, c'_1, c'_2 are sampled from intervals of length C , while k, k' are sampled from intervals of length K .

Fix a first sample (k, c_1, c_2) and set $M = kc_1 + c_2$. A second sample gives a collision exactly when $k'c'_1 + c'_2 = M$. For a fixed c'_1 , the value c'_2 must satisfy $c'_2 \equiv M \pmod{c'_1}$. Since c'_2 ranges over an interval of length C and $c'_1 \geq C$, there is at most one valid c'_2 . Once c'_2 is fixed, $k' = (M - c'_2)/c'_1$ is uniquely determined, and the requirement that k' lies in the allowed interval

can only reduce the number of valid choices. Thus, for any fixed first sample, there are at most C colliding second samples among KC^2 possible second samples, so $\Pr[d = d' \mid k, c_1, c_2] \leq 1/(KC)$.

By the law of total probability,

$$\Pr[d = d'] = \sum_{k, c_1, c_2} \Pr[d = d' \mid k, c_1, c_2] \Pr[k, c_1, c_2] \leq \frac{1}{KC}.$$

Substituting $K = 2^{5\lambda/4}$ and $C = 2^{\lambda/2}$ gives

$$\Pr[d = d'] \leq \frac{1}{2^{5\lambda/4} \cdot 2^{\lambda/2}} = 2^{-7\lambda/4}.$$

Thus, the probability that the extracted endomorphism is scalar is at most $2^{-7\lambda/4}$. $\square$

Definition 8 (Knowledge soundness with knowledge error). *Let $\Pi = (P, V)$ be an interactive proof for a relation R . We say that Π is knowledge sound with knowledge error κ if there exists an expected polynomial-time extractor $\mathcal{E}$ such that, for every possibly dishonest prover P^* and every statement x , whenever $\rho := \Pr[(P^*, V)(x) = 1] \geq \kappa(|x|)$, it holds that*

$$\Pr \left[(x, w) \in R : w \leftarrow \mathcal{E}^{P^*}(x) \right] \geq \rho - \kappa(|x|).$$

Our protocol satisfies 2-special soundness with extraction error δ , where Lemma 2 gives $\delta \leq 2^{-7\lambda/4}$. Let $N = |\mathcal{C}|$ be the size of the challenge space, and let $\rho := \Pr[(P^*, V)(x) = 1]$ be the accepting probability of a possibly dishonest prover P^* . By the tight tree-finding analysis of $(k_1, \dots, k_\mu)$ -special soundness [ACK21], the knowledge extractor in our 3-move case succeeds with probability at least

$$(1 - \delta) \frac{N}{N - 1} \left(\rho - \frac{1}{N} \right).$$

Thus, under the above definition, the corresponding knowledge error is $\kappa = \max\{1/N, \delta\}$. In our parameter setting, $N = 2^\lambda$, and hence $\delta \leq 2^{-7\lambda/4} < 2^{-\lambda} = 1/N$. Therefore, $\kappa = 1/N = 2^{-\lambda}$. The extraction error is thus dominated by the inherent challenge-guessing term $1/N$.

6.3 Hint-Assisted Weak Honest-Verifier Zero Knowledge

We now formulate the hint-assisted wHVZK notion used in our proof and instantiate it with the simulated hint distribution $\mathcal{H}_E^{\text{sim}}$ from Experiment 1. The hints provide the simulator with the response data needed to generate accepting transcripts without using the secret key.

Definition 9 (Hint-assisted wHVZK). *Let $\Sigma = (\mathcal{P}, \mathcal{V})$ be a Σ -protocol for a relation R , and let $\mathcal{H}$ be a hint distribution for R . We say that Σ is $\mathcal{H}$ -hint-assisted wHVZK if there exists a PPT simulator Sim such that, for every $r = \text{poly}(\lambda)$ and every PPT distinguisher $\mathcal{A}$, the advantage of distinguishing r real transcripts of Σ from r simulated transcripts generated by Sim using r hints is negligible in λ . We define this advantage as*

$$\text{Adv}_{\Sigma, \mathcal{H}, \text{Sim}}^{\text{hint-wHVZK}}(\mathcal{A}, r) := |\Pr[\mathcal{A}(T_{\text{real}}) = 1] - \Pr[\mathcal{A}(T_{\text{sim}}) = 1]|,$$

where T_{real} denotes the distribution of r honest transcripts and T_{sim} denotes the distribution of

r transcripts output by Sim with hints from $\mathcal{H}$.

For our protocol, the simulator takes as input the public key and a hint sampled from $\mathcal{H}_{E_{\text{pk}}}^{\text{sim}}$. The first isogeny in the hint determines the simulated commitment, while the second isogeny determines the auxiliary curve and torsion data in the same format as the signing algorithm.

Algorithm 6.1 $\text{Sim}_{\text{Tri}}(\text{pk}, h)$

Input: The public key $\text{pk} = (E_{\text{pk}}, \text{hint}_{\text{pk}})$ and a hint $h = (c_1, c_2, \psi_1, \psi_2) \leftarrow \mathcal{H}_{E_{\text{pk}}}^{\text{sim}}$, where $\psi_1 : E_{\text{pk}} \rightarrow E_{\text{com}}$ and $\psi_2 : E_{\text{com}} \rightarrow E_{\text{aux}}$.

Output: A simulated transcript $(\text{com}, \text{chl}, \text{rsp})$.

- 1: $\text{com} := E_{\text{com}} := \text{The codomain of } \psi_1$
 - 2: $\text{chl} := (c_1, c_2)$
 - 3: $E_{\text{aux}} := \text{The codomain of } \psi_2$
 - 4: $d := \deg(\psi_1)$
 - 5: $(P_{\text{pk}}, Q_{\text{pk}}) := \text{TorsionBasisFromHint}(E_{\text{pk}}, 2^f, \text{hint}_{\text{pk}})$
 - 6: $P_{\text{aux}} := (\psi_2 \circ \psi_1)(P_{\text{pk}})$
 - 7: $Q_{\text{aux}} := (\psi_2 \circ \psi_1)(Q_{\text{pk}})$
 - 8: $\text{rsp} := (E_{\text{aux}}, P_{\text{aux}}, Q_{\text{aux}}, d)$
 - 9: **return** $(\text{com}, \text{chl}, \text{rsp})$
-

The simulator outputs the response in the same format as the signing algorithm. It first recovers the public torsion basis from hint_{pk} and then applies the composed isogeny $\psi_2 \circ \psi_1$ to obtain P_{aux} and Q_{aux} . The truncation by $\text{cff} = 2^{f-e_{\text{rsp}}}$ is left to the verification procedure, as in the real scheme.

Theorem 2 ($\mathcal{H}_E^{\text{sim}}$ -hint-assisted wHVZK). *Let Sim_{Tri} be the simulator defined in Algorithm 6.1. For any $s = \text{poly}(\lambda)$ and any PPT adversary $\mathcal{A}$ against the $\mathcal{H}_E^{\text{sim}}$ -hint-assisted wHVZK experiment with s transcripts, there exists a PPT algorithm $\mathcal{D}$ such that*

$$\text{Adv}_{\Sigma, \mathcal{H}_E^{\text{sim}}, \text{Sim}_{\text{Tri}}}^{\text{hint-wHVZK}}(\mathcal{A}, s) \leq \text{Adv}^{s\text{-hint-dist}}(\mathcal{D}) + \frac{s}{\sqrt{p}}.$$

Proof. Let T_{real} be the distribution of s honest transcripts, and let T_{sim} be the distribution of s transcripts generated by Sim_{Tri} using hints from $\mathcal{H}_E^{\text{sim}}$. We compare them through two hybrid steps.

The first hybrid replaces the honest commitment distribution by the stationary distribution. In the honest protocol, the commitment curve is obtained from an ideal of norm D_{mix} , where D_{mix} is the smallest prime greater than $2^{4\lambda}$. Thus the commitment step corresponds to a one-step walk in the D_{mix} -isogeny graph. By the commitment-mixing bound of Aardal et al. [ABF⁺25, Proposition 2.4], the statistical distance between the honest commitment distribution D_{com} and the stationary distribution S_j is at most $\frac{1}{2}\sqrt{p/D_{\text{mix}}}$. Since $D_{\text{mix}} > p^2$, replacing s honest commitments by stationary ones changes the distinguishing probability by at most $s/\sqrt{p}$.

After this replacement, the commitment distribution agrees with the simulated one. The challenge distribution is also identical, since both experiments sample (c_1, c_2) uniformly from $\mathcal{C}$. Hence the only remaining difference is the response isogeny. In the real distribution, it is

$\varphi_{\text{rsp}} : E_{\text{pk}} \rightarrow E_{\text{com}}$ induced by $I_{\text{rsp}} \leftarrow \text{EquivalentSpecialIdeal}(\overline{I_{\text{sk}}}I_{\text{com}}, c_1, c_2)$. In the simulated distribution, it is the first hint isogeny $\psi_1 : E_{\text{pk}} \rightarrow E_{\text{com}}$ sampled from $\mathcal{H}_{E_{\text{pk}}}^{\text{sim}}$.

Given the response isogeny, the subsequent auxiliary isogeny and auxiliary torsion data are generated by the same efficient rule in both hybrids. More precisely, the auxiliary isogeny is sampled according to the prescribed auxiliary-isogeny distribution, and the points $P_{\text{aux}}, Q_{\text{aux}}$ are computed from the composed isogeny exactly as in Algorithm 6.1. Therefore, any distinguisher between the second hybrid and the simulated transcript distribution yields a distinguisher for the s -hint-dist problem. This bounds the second hybrid step by $\text{Adv}^{s\text{-hint-dist}}(\mathcal{D})$.

Combining the commitment-replacement loss and the response-isogeny indistinguishability loss gives

$$\text{Adv}_{\Sigma, \mathcal{H}_E^{\text{sim}}, \text{Sim}_{\text{Tr}}}^{\text{hint-wHVZK}}(\mathcal{A}, s) \leq \text{Adv}^{s\text{-hint-dist}}(\mathcal{D}) + \frac{s}{\sqrt{p}}.$$

This proves the theorem. $\square$

6.4 EUF-CMA Security

We now apply the Fiat–Shamir-with-hints theorem to obtain the EUF-CMA bound.

Theorem 3 ([ABF⁺25, Theorem 1]). *Let $(R, \mathcal{H})$ be a relation with hints, and let Σ be a Σ -protocol for R with challenge space $\mathcal{C}$. Assume that Σ is special-sound with respect to a soundness relation $\tilde{R}$, and let Sim be an $\mathcal{H}$ -hint-assisted simulator for Σ . For any PPT adversary $\mathcal{A}$ against the EUF-CMA security of the Fiat–Shamir signature scheme $\text{SIG}[\Sigma]$ making at most q random-oracle queries and s signing queries, there exist an expected polynomial-time algorithm $\mathcal{E}$ and a PPT algorithm $\mathcal{B}$ such that*

$$\begin{aligned} \text{Adv}_{\text{SIG}[\Sigma]}^{\text{EUF-CMA}}(\mathcal{A}) &\leq (q+1) \left(\text{Adv}_{(\tilde{R}, \mathcal{H})}^{\text{hint-rel}}(\mathcal{E}, s) + \text{Adv}_{\Sigma, \mathcal{H}, \text{Sim}}^{\text{hint-wHVZK}}(\mathcal{B}, s) + \frac{1}{|\mathcal{C}|} \right) \\ &\quad + (q+s+1)s \cdot \text{MinEnt}(\Sigma). \end{aligned}$$

Here $\text{MinEnt}(\Sigma)$ denotes the maximum point probability of the honest commitment distribution.

We instantiate the abstract terms in Theorem 3 with the one-endomorphism relation, the simulated hint distribution, and the hint-assisted wHVZK theorem proved above.

Theorem 4 (EUF-CMA security). *For any PPT adversary $\mathcal{A}$ against the EUF-CMA security of the Fiat–Shamir signature scheme making at most q random-oracle queries and s signing queries, there exist PPT algorithms $\mathcal{B}$ and $\mathcal{D}$ such that*

$$\begin{aligned} \text{Adv}^{\text{EUF-CMA}}(\mathcal{A}) &\leq (q+1) \left(\text{Adv}^{s\text{-hint-OneEnd}_p}(\mathcal{B}) + \text{Adv}^{s\text{-hint-dist}}(\mathcal{D}) + \frac{s}{\sqrt{p}} + 2^{-\lambda} \right) \\ &\quad + (q+s+1)s \cdot \frac{15}{p}. \end{aligned}$$

Proof. We apply Theorem 3 to the underlying Σ -protocol with hint distribution $\mathcal{H}_E^{\text{sim}}$. In our case, the soundness relation $\tilde{R}$ is the one-endomorphism relation: given a supersingular elliptic curve E , a valid witness is an element of $\text{End}(E) \setminus \mathbb{Z}$ in efficient representation. Therefore, the

hinted hard-relation term in Theorem 3 is instantiated by the s -hint-OneEnd $_p$ problem, namely

$$\mathbf{Adv}_{(\tilde{R}, \mathcal{H}_E^{\text{sim}})}^{\text{hint-rel}}(\mathcal{E}, s) = \mathbf{Adv}^{s\text{-hint-OneEnd}_p}(\mathcal{B})$$

for a suitable PPT algorithm $\mathcal{B}$.

The challenge space is $\mathcal{C} = \{2^{\lambda/2}, 2^{\lambda/2} + 1, \dots, 2^{\lambda/2+1}\}^2$. Hence $|\mathcal{C}| \geq 2^\lambda$, and the challenge-guessing term satisfies $1/|\mathcal{C}| \leq 2^{-\lambda}$.

By Theorem 2, the hint-assisted wHVZK term induced by the reduction is bounded by

$$\mathbf{Adv}_{\Sigma, \mathcal{H}_E^{\text{sim}}, \text{Sim}_{\text{Tri}}}^{\text{hint-wHVZK}}(\mathcal{B}_{\text{zk}}, s) \leq \mathbf{Adv}^{s\text{-hint-dist}}(\mathcal{D}) + \frac{s}{\sqrt{p}},$$

for some PPT algorithms $\mathcal{B}_{\text{zk}}$ and $\mathcal{D}$. The term $s/\sqrt{p}$ is the statistical loss incurred when the real commitment distribution is replaced by the stationary distribution.

Finally, by Lemma 1, the protocol has α -min-entropy with $\alpha \geq \log_2 p - \log_2(15)$. Hence $\text{MinEnt}(\Sigma) = 2^{-\alpha} \leq 15/p$, so the min-entropy contribution in Theorem 3 is bounded by $(q + s + 1)s \cdot 15/p$.

Substituting these bounds into Theorem 3 gives the claimed inequality. $\square$

Remark on signing limits. In Theorem 4, the query-dependent loss terms are the commitment-replacement term $(q+1)s/\sqrt{p}$, the challenge-guessing term $(q+1)2^{-\lambda}$, and the min-entropy term $(q + s + 1)s \cdot 15/p$. The factor $15/p$ follows from Lemma 1, where $D_{\text{mix}} > 2^{4\lambda}$ and $p \approx 2^{2\lambda}$ imply $D_{\text{mix}} > p^2$ and hence $\Pr[E_{\text{com}} = E^*] \leq 15/p$.

For the $\lambda = 160$ instance, we have $p \approx 2^{320}$ and $\sqrt{p} \approx 2^{160}$. If a single key pair is used to sign $s = 2^{64}$ distinct messages and the adversary makes at most $q = 2^{80}$ random-oracle queries, then the commitment-replacement term is approximately $2^{80} \cdot 2^{64} \cdot 2^{-160} = 2^{-16}$. Hence the present $1/\sqrt{p}$ commitment-replacement bound is not sufficient to give an 80-bit concrete security margin for this query regime.

This loss can be reduced by increasing D_{mix} . In the general commitment-mixing estimate, the replacement loss is of order $\sqrt{p/D_{\text{mix}}}$. To make $(q+1)s\sqrt{p/D_{\text{mix}}} \leq 2^{-80}$ for $q \leq 2^{80}$ and $s = 2^{64}$, it is enough to choose $D_{\text{mix}} \geq 2^{2\lambda+448}$, which gives $D_{\text{mix}} \geq 2^{768}$ for $\lambda = 160$. This is larger than the current choice $D_{\text{mix}} \approx p^2 \approx 2^{640}$.

A cleaner and stronger choice is to enforce a $1/p$ commitment-replacement bound by taking $D_{\text{mix}} \geq p^3 \approx 2^{6\lambda}$, namely $D_{\text{mix}} \geq 2^{960}$ for $\lambda = 160$. Then the commitment-replacement term becomes roughly $(q+1)s/p$, which is at most $2^{80} \cdot 2^{64}/2^{320} = 2^{-176}$.

After this improvement, the other terms in Theorem 4 are also compatible with the 80-bit target: for $q \leq 2^{80}$ and $\lambda = 160$, the challenge-guessing term is approximately 2^{-80} , and the min-entropy term is of order $15 \cdot 2^{-176}$. Therefore, with the increased D_{mix} described above, the $\lambda = 160$ instance can support 2^{64} signing queries under the assumption $q \leq 2^{80}$ within this concrete reduction.

It is worth noting that increasing D_{mix} does not introduce additional online signing cost in our scheme. The parameter D_{mix} only affects the offline commitment-generation parameter, while the online signing procedure remains unchanged.

7 Performance Evaluation

7.1 Design-Level Performance Analysis

At the representation level, SQIsignTriangle keeps the same compact-key design principle as SQIsign: a public key is essentially a supersingular elliptic curve together with a small hint for reconstructing a torsion basis. Its key generation algorithm is the same as the SQIsign key generation algorithm used in the submitted implementation, and the commitment generation inside signing is also the same as the SQIsign commitment generation procedure. The difference lies in the challenge and response mechanism. Instead of asking the prover to answer a fixed degree challenge isogeny, the verifier derives two challenge integers c_1, c_2 , and the prover returns a response isogeny of degree q satisfying

$$q \equiv c_2 \pmod{c_1}.$$

This modular-degree challenge is the main design choice that changes the performance profile of SQIsignTriangle relative to SQIsign.

Key generation follows the SQIsign procedure: it samples and reduces a secret ideal, translates it into an isogeny, evaluates the images of the 2^f -torsion basis, and stores the public-basis hint and change-of-basis matrix. Its running time is dominated by the ideal-to-isogeny translation and torsion-image evaluation. The ideal sampling, basis hint generation, basis-change computation, and serialization-related work are comparatively small overheads.

Signing has three conceptual phases: commitment, challenge, and response. The commitment phase is exactly the SQIsign commitment phase and is one of the dominant costs, because it requires an ideal-to-isogeny evaluation and torsion basis evaluation. The challenge phase only computes curve invariants, applies SHAKE256, and tests small prime candidates; it is fast compared with isogeny evaluation. The response phase is the main new component of SQIsignTriangle : it uses [EquivalentSpecialIdeal](#) to find a response degree q satisfying $q \equiv c_2 \pmod{c_1}$, and then evaluates an auxiliary ideal-to-isogeny map to produce the compact auxiliary torsion data. The auxiliary ideal-to-isogeny evaluation is the other dominant cost in signing, while the response search contributes most of the signing-time variance.

Verification consists of public-basis reconstruction, basis scaling, a two-dimensional isogeny-chain computation, and a final hash/congruence check. The dominant cost is the computation of the $(2, 2)$ -isogeny chain from the Kani kernel, whose length is controlled by $e_{\text{rsp}} = \lceil \log_2 q \rceil$. Basis reconstruction and scaling have moderate cost, while input validation, hash recomputation, and the congruence check are fast. Consequently, verification is much more stable than signing and is roughly linear in e_{rsp} measured in $(2, 2)$ -isogeny steps.

Therefore, at the design level, concrete performance is governed mainly by the number of ideal-to-isogeny translations and the length of the verification $(2, 2)$ -isogeny chain. Hashing, modular congruence tests, curve copies, and byte serialization are fast secondary costs.

Table 7 summarizes the resulting performance profile at the design level.

Table 7: Design-level performance drivers of SQIsignTriangle .

Operation	Main time-consuming steps	Reason for the cost
KeyGen	Ideal-to-isogeny translation and torsion-image evaluation.	These are the expensive parts of the SQIsign key-generation procedure.
Sign	Commitment ideal-to-isogeny evaluation and auxiliary ideal-to-isogeny evaluation.	Signing performs two costly explicit isogeny evaluations; response retries add variance.
Verify	The $(2, 2)$ -isogeny chain from the Kani kernel.	Its length is controlled by e_{rsp} , so verification cost is essentially linear in e_{rsp} .

7.2 Parameter Selection

Except for the Fiat–Shamir challenge derivation described below, SQIsignTriangle uses the same concrete parameter-selection principles and the same underlying SQIsign parameter sets as the SQIsign specification. For each security level, the base field is chosen as

$$p = c \cdot 2^f - 1,$$

where the large power of two in $p + 1$ provides the full 2^f -torsion used by the two-dimensional isogeny computations. The constants for the starting curve, the quaternion order, the torsion bases, and the auxiliary precomputations are generated by the SQIsign precomputation scripts and are included in the C implementation. The four submitted parameter sets are shown in Table 8.

Table 8: SQIsignTriangle parameter sets inherited from SQIsign.

Security level λ	Field prime p	f	$\lceil \log_2 p \rceil$	$\mathbb{F}_p$ bytes
128	$5 \cdot 2^{248} - 1$	248	251	32
160	$9 \cdot 2^{309} - 1$	309	313	40
256	$27 \cdot 2^{500} - 1$	500	505	64
512	$113 \cdot 2^{1016} - 1$	1016	1023	128

The hash-to-challenge routine is specific to SQIsignTriangle . In the submitted C implementation, the function `hash_to_challenge_prime_with_pk_j` first encodes the public j -invariant and the commitment-curve j -invariant, absorbs these encodings and the message into incremental SHAKE256, and then repeatedly squeezes two candidate integers. Each candidate has $\lambda/2$ bits, namely 8, 10, 16, and 32 bytes for security levels $\lambda = 128, 160, 256$, and 512. The first candidate among the two that is congruent to 3 modulo 4 and passes 40 Miller–Rabin primality-test rounds is used as the prime challenge modulus c_1 ; the other half of the same SHAKE256 block is used as c_2 . Signing then constructs an odd response degree q such that

$$q \equiv c_2 \pmod{c_1},$$

and verification recomputes the same challenge from the recovered commitment curve and checks

this congruence.

7.3 Key and Signature Sizes

The public key consists of the public elliptic curve and a one-byte basis hint. The response contains the auxiliary curve, the compressed auxiliary torsion basis, and the response degree q . The submitted C implementation serializes the auxiliary basis by four scalar coefficients and serializes q in the same short length as these coefficients. The resulting byte lengths in the submitted C signature API are shown in Table 9.

Table 9: SQIsignTriangle encoded key and signature sizes in the submitted C API.

Security level λ	Public key	Secret key	Signature	Aux. basis coeff.	q bytes
128	65	353	204	28	28
160	81	437	255	35	35
256	129	701	408	56	56
512	257	1409	816	112	112

7.4 Implementation Platforms

The submission package contains three implementation tracks.

- A portable reference C11 implementation following the C signature API. It contains the full key generation, signing and verification algorithms, uses Qlapoti for the ideal-to-isogeny translation, and includes self-tests, known-answer-test, and CPU-cycle benchmarks.
- An optimized 64-bit x86 implementation in the same C code base. The measured build uses `SQISIGN_BUILD_TYPE=broadwell`, with assembly-optimized finite-field arithmetic for Intel Broadwell-compatible processors and later.
- A SageMath implementation used to validate the mathematical correctness of the construction and to prototype the `EquivalentSpecialIdeal` and verification logic. It is not intended as the primary performance implementation.

The benchmark data in Table 10 were obtained from the optimized C implementation on a 64-bit PC processor:

- processor: 13th Gen Intel(R) Core(TM) i7-13700, x86-64, 24 logical CPUs, with AVX2, BMI2, ADX, AES-NI, and SHA-NI available;
- memory and operating environment: 15 GiB RAM, Ubuntu 24.04.3 LTS under WSL2;
- compiler and build system: GCC 13.3.0, CMake 3.28.3, GNU Make 4.3;
- build options: `CMAKE_BUILD_TYPE=Release`, `SQISIGN_BUILD_TYPE=broadwell`, `ENABLE_SIGN=ON`, `GMP_LIBRARY=SYSTEM`, `-O3`, system GMP, and 64-bit field radix.

7.5 Software Performance

The software benchmarks use the `apps/benchmark_lvl11`, `apps/benchmark_lvl12`, `apps/benchmark_lvl15`, and `apps/benchmark_lvl16` programs from the C implementation. Each run signs a 32-byte message and reports CPU cycles for key generation, signing and verification. The current measurements were obtained with `--iterations=10` for each security level and are therefore intended as initial engineering measurements; more extensive benchmark campaigns can reduce the variance of the key generation and signing, which contain randomized rejection-sampling steps.

Table 10: SQIsignTriangle optimized C performance in 10^6 CPU cycles on Intel Core i7-13700. Results are over 10 benchmark runs.

λ	Operation	Average	Std. dev.	Median	Min.	Max.
128	KeyGen	13.903	4.479	12.337	8.920	20.401
	Sign	37.276	10.676	36.748	27.899	64.687
	Verify	4.029	0.324	3.866	3.802	4.856
160	KeyGen	21.813	8.130	20.342	15.568	44.295
	Sign	63.981	21.123	68.299	37.895	104.540
	Verify	7.568	0.121	7.566	7.419	7.737
256	KeyGen	108.494	48.536	103.068	54.355	210.166
	Sign	201.325	30.253	194.313	151.426	253.972
	Verify	27.725	3.442	26.071	25.336	37.107
512	KeyGen	783.312	351.761	674.666	502.816	1564.300
	Sign	2698.805	1232.049	3334.319	1078.011	4816.004
	Verify	258.839	11.367	263.993	245.386	277.432

7.6 Software Resource Consumption

We measured the peak resident set size of the benchmark process using `/usr/bin/time -v`. These figures include the benchmark harness, the C API buffers for 10 iterations, the linked GMP runtime, and all temporary memory allocated by the implementation during key generation, signing and verification.

Table 11: SQIsignTriangle software resource consumption measured with `/usr/bin/time -v`.

Security level λ	Iterations	Peak resident set size	Wall-clock time
128	10	3072 KiB	0.38 s
160	10	3072 KiB	0.48 s
256	10	3072 KiB	1.53 s
512	10	3456 KiB	19.97 s

8 Feature Statements

In this section, we mainly demonstrate the features of the SQIsignTriangle signature scheme.

- ⊕ **Innovativeness.** SQIsignTriangle introduces a novel approach to post-quantum isogeny-based signatures by shifting from traditional 2-isogenies chain challenges to a flexible modular degree constraint ($q \equiv c_2 \pmod{c_1}$). This elegant relaxation efficiently resolves the 1-bit challenge space limitation of early identification protocols. As a result, the scheme maintains exceptionally compact key and signature sizes, significantly enriching the current landscape of practical PQC candidates
- ⊕ **Strong Security.** SQIsignTriangle is constructed upon a highly rigorous 3-round Σ -protocol compiled via the Fiat-Shamir with hints paradigm. It achieves strong EUF-CMA security strictly grounded in the Endomorphism Ring Problem. Notably, the scheme features tight security reductions with meticulous bounds on commitment min-entropy. By strictly bounding the parameters and tracking the exact size of distributions, the proof rigorously avoids statistical indistinguishability fallacies that often arise when analyzing the coverage of random isogeny walks.
- ⊕ **Simplicity.** This isogeny-based signature scheme has a very simple and understandable structure: it is obtained from a two-round identification protocol via the hash-and-sign method. Moreover, the mathematical theory is relatively simple and not complicated.
- ⊕ **Constant-time performance.** This signature scheme can be performed in constant time, effectively mitigating certain types of side-channel attacks such as timing attacks. In addition, the scheme is relatively straightforward to implement in practice, requiring no complex cryptographic primitives or heavy computational overhead.
- ⊕ **Compatibility** This signature scheme adopts the same prime field parameter as SQIsign, which provides strong compatibility with existing SQIsign-oriented software and hardware environments. Since the underlying finite-field arithmetic is defined over the same prime, the core arithmetic modules can be reused or adapted with minimal modification. This design also makes the implementation portable across different platforms, including general-purpose CPUs, embedded devices, and hardware acceleration frameworks. As a result, our scheme can be integrated into existing SQIsign-compatible toolchains with low engineering overhead, while maintaining good flexibility for deployment under diverse software and hardware conditions.

References

- [AAA⁺25] Marius A. Aardal, Gora Adj, Diego F. Aranha, Andrea Basso, Isaac Andrés Canales Martínez, Jorge Chávez-Saab, Maria Corte-Real Santos, Pierrick Dartois, Luca De Feo, Max Duparc, Jonathan Komada Eriksen, Tako Boris Fouotsa, Décio Luiz Gazzoni Filho, Basil Hess, David Kohel, Antonin Leroux, Patrick Longa, Luciano Maino, Michael Meyer, Kohei Nakagawa, Hiroshi Onuki, Lorenz Panny, Sikhar Patranabis, Christophe Petit, Giacomo Pope, Krijn Reijnders, Damien Robert, Francisco

- Rodríguez-Henríquez, Sina Schaeffler, and Benjamin Wesolowski. SQIsign. Technical report, National Institute of Standards and Technology, 2025.
- [AAF⁺26] Marius A. Aardal, Diego F. Aranha, Yansong Feng, Yiming Gao, and Yanbin Pan. Better bounds for finding fixed-degree isogenies via coppersmith’s method. In *Advances in Cryptology – EUROCRYPT 2026*, Lecture Notes in Computer Science. Springer, 2026. To appear.
- [ABF⁺25] Marius A. Aardal, Andrea Basso, Luca De Feo, Sikhar Patranabis, and Benjamin Wesolowski. A complete security proof of sqsign. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part VI*, Lecture Notes in Computer Science, pages 190–222. Springer, 2025.
- [ACK21] Thomas Attema, Ronald Cramer, and Lisa Kohl. A compressed varsigma -protocol theory for lattices. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021*, pages 549–579, Cham, 2021. Springer International Publishing.
- [BBC⁺25] Andrea Basso, Giacomo Borin, Wouter Castryck, Maria Corte-Real Santos, Riccardo Invernizzi, Antonin Leroux, Luciano Maino, Frederik Vercauteren, and Benjamin Wesolowski. PRISM: simple and compact identification and signatures from large prime degree isogenies. In Tibor Jager and Jiaxin Pan, editors, *Public-Key Cryptography - PKC 2025 - 28th IACR International Conference on Practice and Theory of Public-Key Cryptography, Røros, Norway, May 12-15, 2025, Proceedings, Part III*, volume 15676 of *Lecture Notes in Computer Science*, pages 300–332. Springer, 2025.
- [BDF⁺24] Andrea Basso, Pierrick Dartois, Luca De Feo, Antonin Leroux, Luciano Maino, Giacomo Pope, Damien Robert, and Benjamin Wesolowski. Sqsign2d-west - the fast, the small, and the safer. In *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part III*, pages 339–370, 2024.
- [BFLS26] Giacomo Borin, Luca De Feo, Guido Maria Lido, and Sina Schaeffler. The SQInstructor: a guide to SQIsign and the deuring correspondence with level structures. Cryptology ePrint Archive, Paper 2026/493, 2026.
- [BKM⁺24] Benjamin Benčina, Péter Kutas, Simon-Philipp Merz, Christophe Petit, Miha Stopar, and Charlotte Weitkämper. Improved algorithms for finding fixed-degree isogenies between supersingular elliptic curves. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part V*, pages 183–217, 2024.
- [BKV19] Ward Beullens, Thorsten Kleinjung, and Frederik Vercauteren. Csi-fish: Efficient isogeny based signatures through class group computations. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information*

- Security, Kobe, Japan, December 8-12, 2019, Proceedings, Part I*, volume 11921 of *Lecture Notes in Computer Science*, pages 227–247. Springer, 2019.
- [BSE⁺25] Giacomo Borin, Maria Corte-Real Santos, Jonathan Komada Eriksen, Riccardo Invernizzi, Marzio Mula, Sina Schaeffler, and Frederik Vercauteren. Qlapoti: Simple and efficient translation of quaternion ideals to isogenies. *Cryptology ePrint Archive*, Paper 2025/1604, 2025.
- [CFL⁺18] Anamaria Costache, Brooke Feigon, Kristin E. Lauter, Maike Massierer, and Anna Puskás. Ramanujan graphs in cryptography. *IACR Cryptol. ePrint Arch.*, page 593, 2018.
- [CS18] Craig Costello and Benjamin Smith. Montgomery curves and their arithmetic: The case of large characteristic fields. *Journal of Cryptographic Engineering*, 8(3):227–240, 2018.
- [DF24] Max Duparc and Tako Boris Fouotsa. Sqprime: A dimension 2 variant of sqsignhd with non-smooth challenge isogenies. In *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part III*, pages 396–429, 2024.
- [DG16] Christina Delfs and Steven D. Galbraith. Computing isogenies between supersingular elliptic curves over $\mathbb{F}_p$. *Des. Codes Cryptogr.*, 78(2):425–440, 2016.
- [DLRW24] Pierrick Dartois, Antonin Leroux, Damien Robert, and Benjamin Wesolowski. Sqsignhd: New dimensions in cryptography. In *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part I*, pages 3–32, 2024.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology—CRYPTO’86*, pages 186–194. Springer Berlin Heidelberg, 1987.
- [GPS17] Steven D. Galbraith, Christophe Petit, and Javier Silva. Identification protocols and signature schemes based on supersingular isogeny problems. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 3–33. Springer, 2017.
- [JAC⁺22] David Jao, Reza Azarderakhsh, Matthew Campagna, Craig Costello, Luca De Feo, Basil Hess, Aaron Hutchinson, Amir Jalali, Koray Karabina, Brian Koziel, Brian LaMacchia, Patrick Longa, Michael Naehrig, Geovandro Pereira, Joost Renes, Vladimir Soukharev, David Urbanik, Gora Adj, Jesus-Javier Chi-Domínguez, Samuel J. L., Jonathan F., Armando Faz-Hernández, Kelsey E. M., Francisco

- Rodríguez-Henríquez, Ana I. S., Gustavo S. M., and Mila S. Supersingular isogeny key encapsulation. National Institute of Standards and Technology, Technology Administration, U.S. Department of Commerce, 2022. Submission to the NIST Post-Quantum Cryptography Standardization Project.
- [Koh96] David Kohel. *Endomorphism rings of elliptic curves over finite fields*. PhD thesis, University of California, Berkeley, 1996.
- [Nat24a] National Institute of Standards and Technology. FIPS 204: Module-Lattice-Based Digital Signature Standard. Technical report, U.S. Department of Commerce, August 2024.
- [Nat24b] National Institute of Standards and Technology. FIPS 205: Stateless Hash-Based Digital Signature Standard. Technical report, U.S. Department of Commerce, August 2024.
- [NO25] Kohei Nakagawa and Hiroshi Onuki. Sqsign2dpush: Faster signature scheme using 2-dimensional isogenies. *IACR Cryptol. ePrint Arch.*, page 897, 2025.
- [NOC⁺24] Kohei Nakagawa, Hiroshi Onuki, Wouter Castryck, Mingjie Chen, Riccardo Invernizzi, Gioella Lorenzon, and Frederik Vercauteren. Sqsign2d-east: A new signature scheme using 2-dimensional isogenies. In *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part III*, pages 272–303, 2024.
- [PW24] Aurel Page and Benjamin Wesolowski. The supersingular endomorphism ring and one endomorphism problems are equivalent. In *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part VI*, pages 388–417, 2024.
- [XLZO25] Zheng Xu, Kaizhan Lin, Chang-An Zhao, and Yi Ouyang. Sqsign2d²: New sqsign2d variant by leveraging power smooth isogenies in dimension one. *IACR Cryptol. ePrint Arch.*, page 920, 2025.